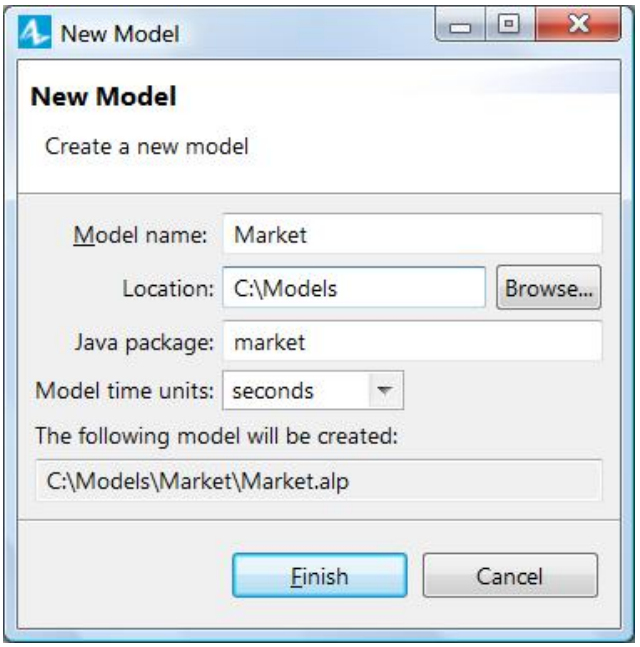
Laboratory 1
Market model

We will build an agent-based model of a consumer market – one where each consumer will be an agent – to help us understand how a product enters the market. Since human decisions always include stochastics, agent based modeling is ideal for modeling market simulations. We assume that the model includes 5000 people who don't use the product, but a combination of advertising and word of mouth will eventually lead them to purchase it.

Phase 1. Creating the agent population

We will start by creating a simple model that depicts how advertising leads consumers to purchase our product. Our model's consumers won't use the product at first, but they are all potentially interested in using it. We will also represent advertising's influence on consumer demand by allowing a specific percentage of them to become interested in purchasing the product during a given day. For our purposes, *Advertising effectiveness = 0.01* determines the percentage of potential users that become ready to buy the product during a given day.

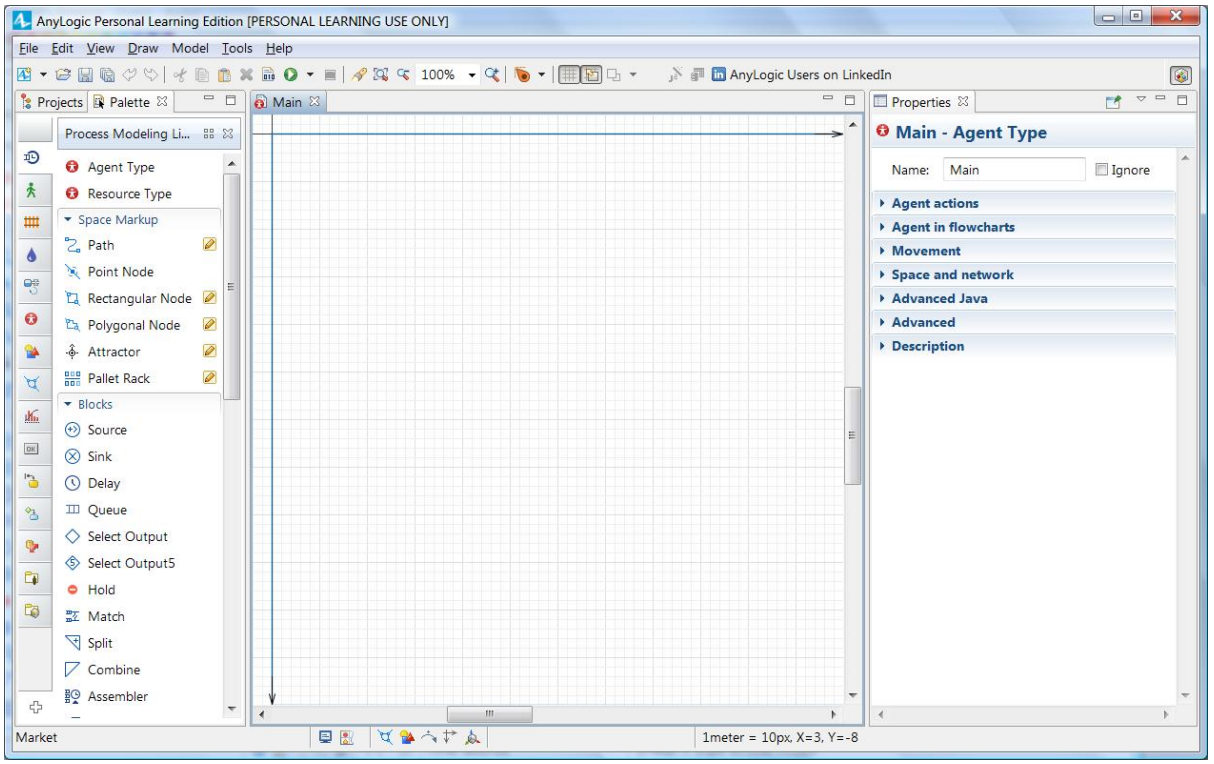
1. Start AnyLogic and the **Welcome** page displays. Close the **Welcome** page, and create a new model by selecting **File** > **New** > **Model** from AnyLogic's main menu. The **New Model** wizard will open.



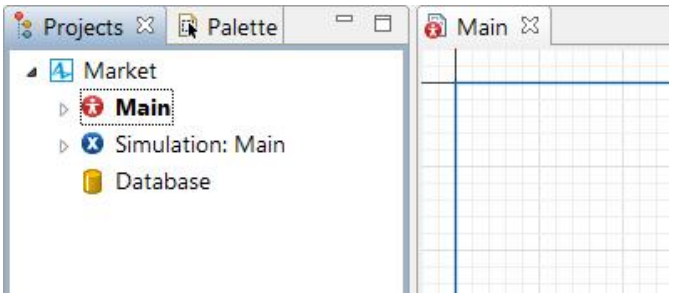
2. In the **Model name** box, enter the new model's name: *Market*.

3. In the **Location** box, select the folder where you want to create the model. You can browse for a folder by clicking **Browse** or type the name of the folder you want to create in the **Location** box.

4. Click **Finish**.



5. We will open the **Projects** view to examine the model's structure. You will find the **Palette** and **Projects** views in the workspace's left section, and you can switch from the **Palette** view to the **Projects** view by clicking the **Projects** tab.



In the graphical editor, you will see the empty diagram of the model's *Main* agent type.

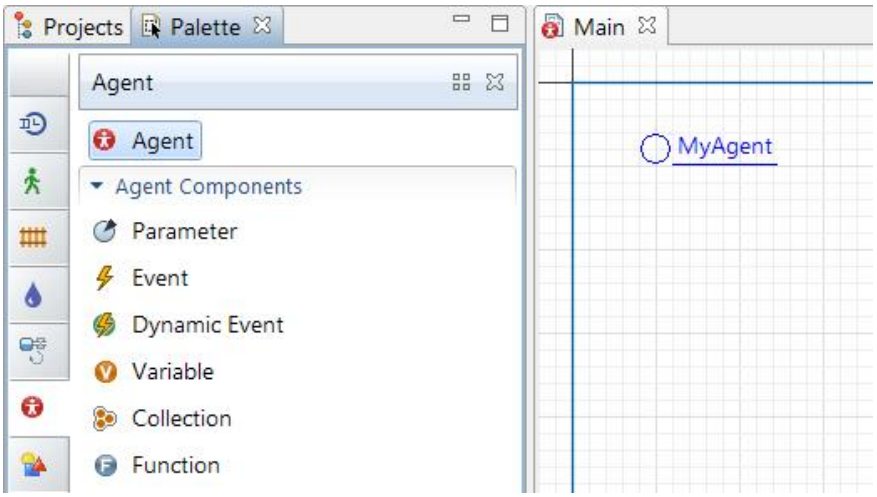
Our model has one agent type, *Main*. To add consumers, we will need to create an agent type to represent consumers, and then create an agent population made up of instances of this consumer agent type. In AnyLogic 7, you can use the helpful **New agent** wizard to create agents.

6. We want to add a new model element, but we first need to switch to the **Palette** view by clicking the **Palette** tab.

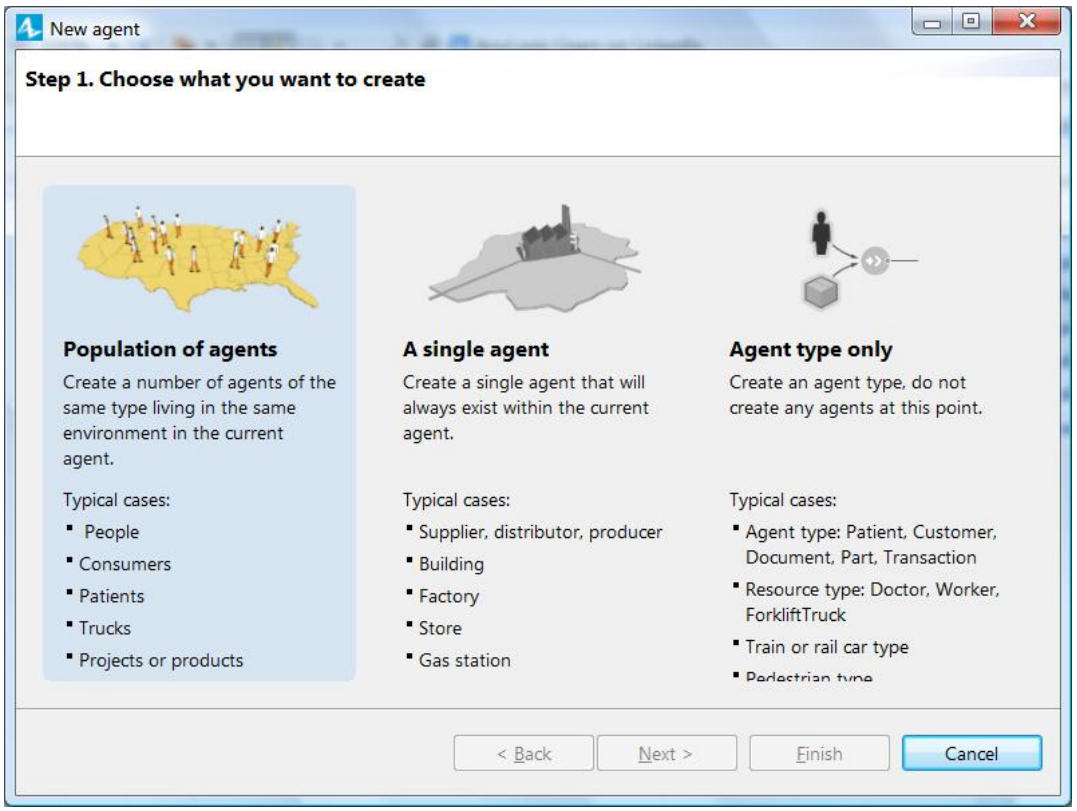
7. Open the **Agent** palette. To open a specific palette, go to the **Palette** view and hover your mouse over the view's vertical navigation panel.

8. It will expand to show the names of all palettes so you can select the one you need. Click the **Agent** palette in the list to select it.

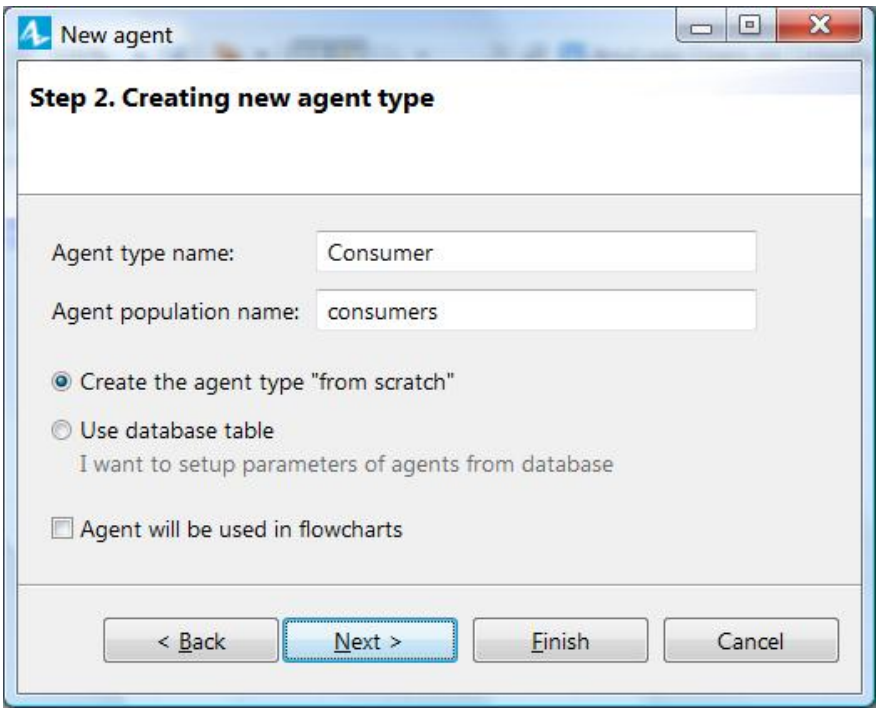
9. Drag the **Agent** from the **Agent** palette on to the *Main* diagram, and the **New agent** wizard will open.



10. On the **Step 1. Choose what you want to create** page, select the option that best meets your needs. Since we want to create multiple agents of the same type, select **Population of agents** and click **Next**.

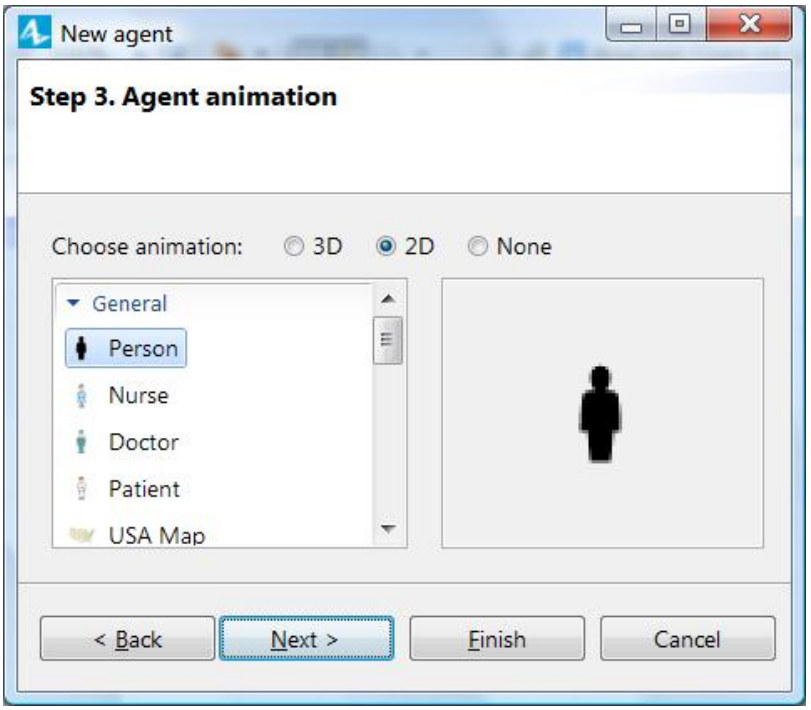


11. On the **Step 2. Creating new agent type** page, in **Agent type name** box, type *Consumer*. The information in the **Agent population name** box will automatically change to *consumers*.



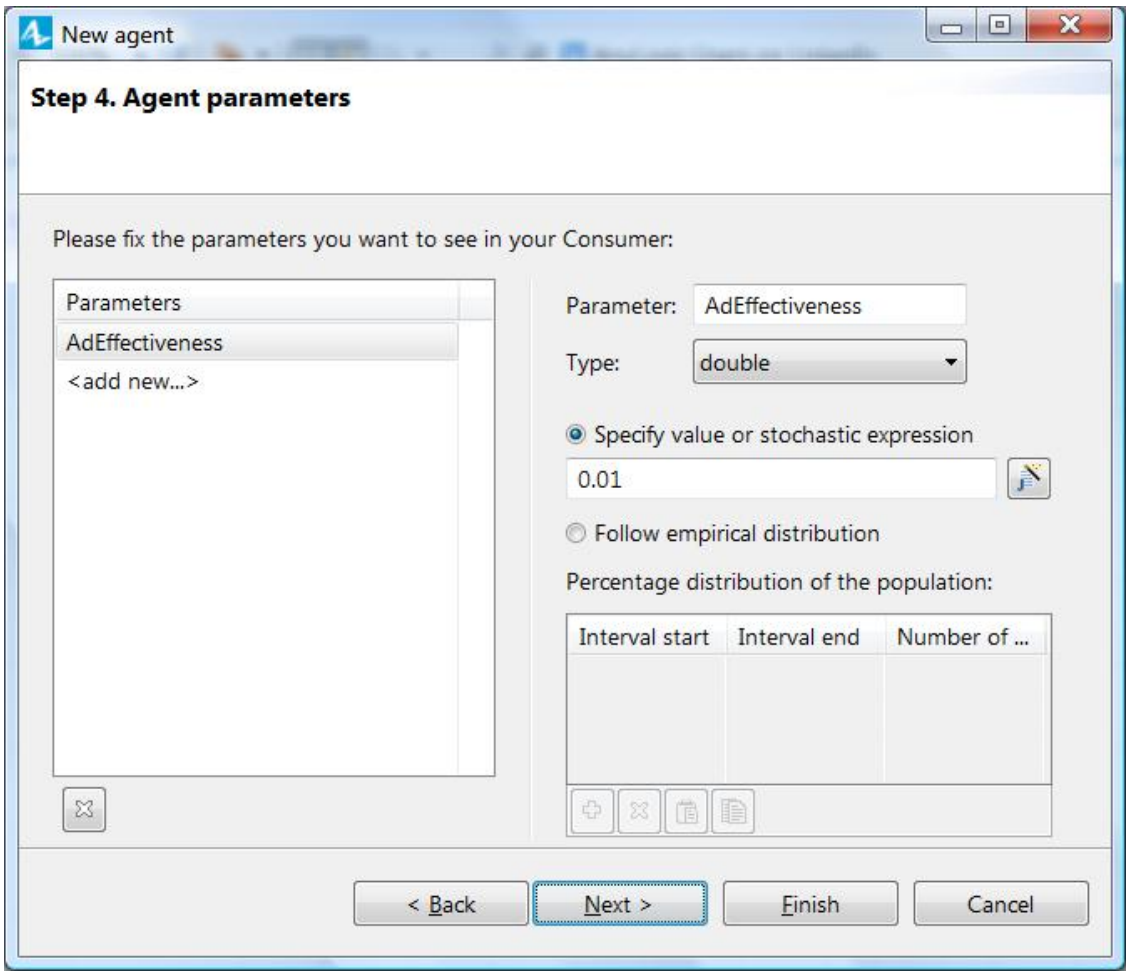
12. Click **Next**.

13. On the **Agent animation** page, choose the agent's animation shape. Since we're creating a simple model that uses 2D animation, choose **2D**, select the General list's first item: **Person**, and click **Next**.



14. On the **Agent Parameters** page, define the agent's parameters or characteristics.

    Since our model only considers advertising-related product purchases, we will add a parameter – *AdEffectiveness* – to define the percentage of potential users who become ready to buy the product during a given day.



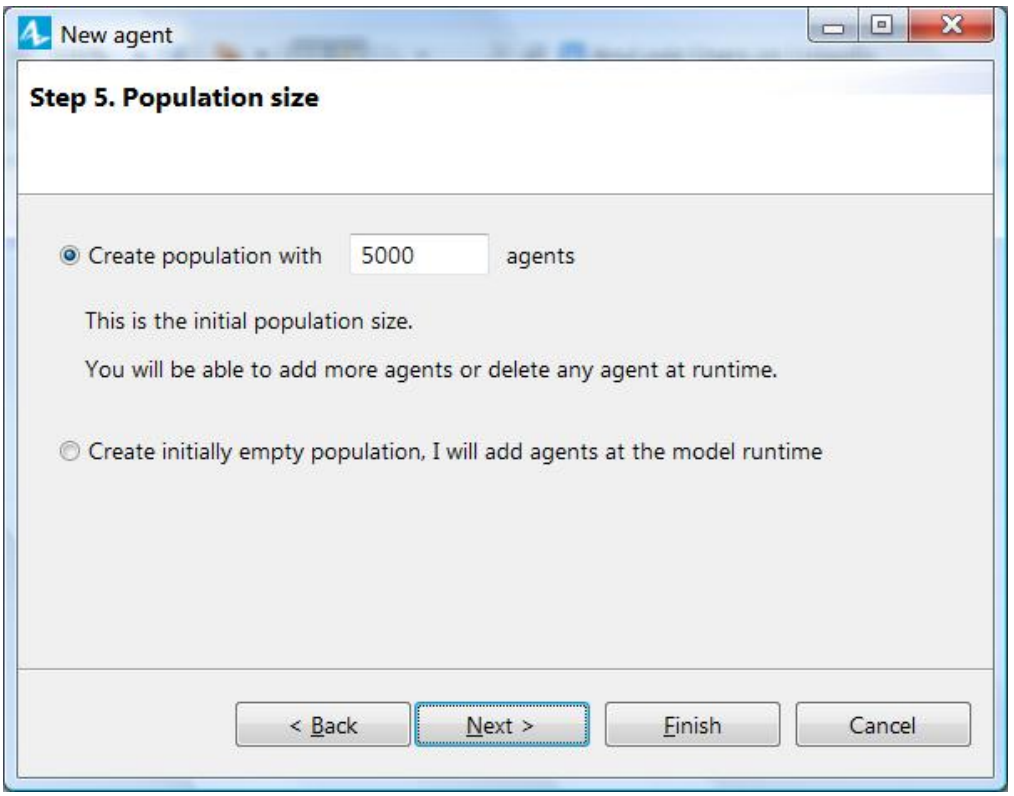
15. On the left section, in the **Parameters** table, click <**add new...**> to create a parameter.

16. In the **Parameter** box, change the default parameter's name to *AdEffectiveness*, and choose **double** as the parameter **Type**. We will assume an average of 1% of our model's potential users will want to buy the product during a given day, so specify *0.01* as the parameter's value.

17. Click **Next**.

18. On the **Population size** page, type 5000 in the **Create population with ... agents** box to create 5000 instances of the *Consumer* type. Each instance in the population will model a specific agent-consumer.

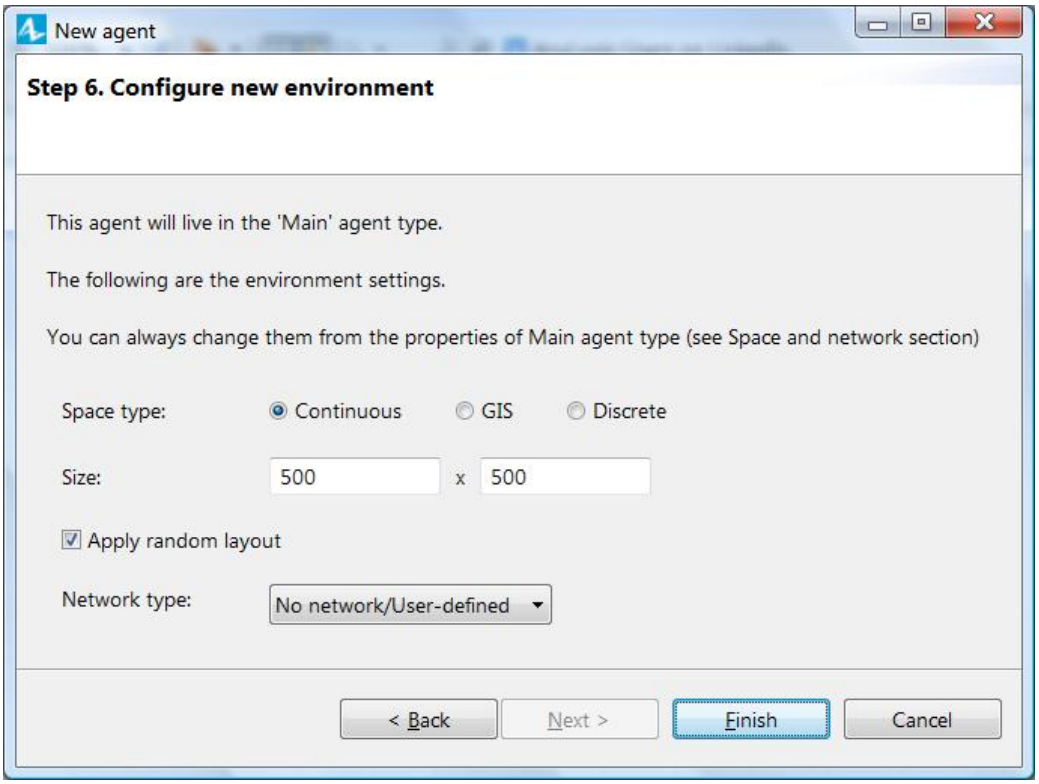
    While we have created our agent population, we won't see 5000 *Person* animation figures on *Main* diagram. Instead, AnyLogic will use the 5000 agents in the population we have called *consumers* to simulate the market when we run our model.



19. Click **Next**.

20. On the **Configure new environment** page, accept the default values for the environment's space type (**Continuous**) and both its **Width** and **Height** values (500). AnyLogic will display the agents in a 500x500 pixel rectangle.

21. Select the **Apply random layout** box to randomly distribute the agents across the 500 pixel width and height we have defined. Since we don't want to create an agent network, we will accept the default **No network/User-defined network** type.

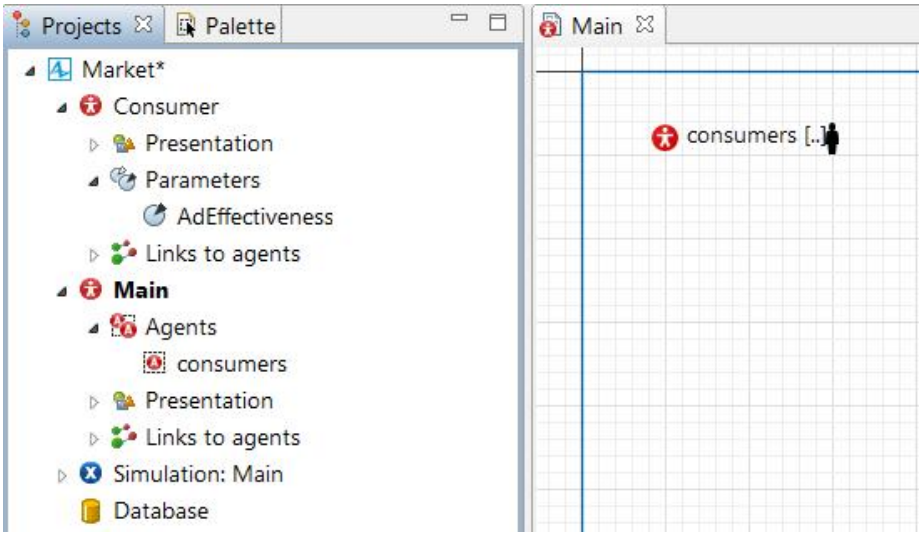


22. Click **Finish**.

23. Use the **Projects** view to see the new elements that the wizard created. Expand the model tree branches to see the internals.

    Our model now has two agent types: *Main* and *Consumer*.

    - The *Consumer* agent type has the agent's animation shape (*person*, in the **Presentation** branch) and the parameter *AdEffectiveness*.
    - The *Main* agent type contains the agent population *consumers* (a set of 5000 agents of type *Consumer*).

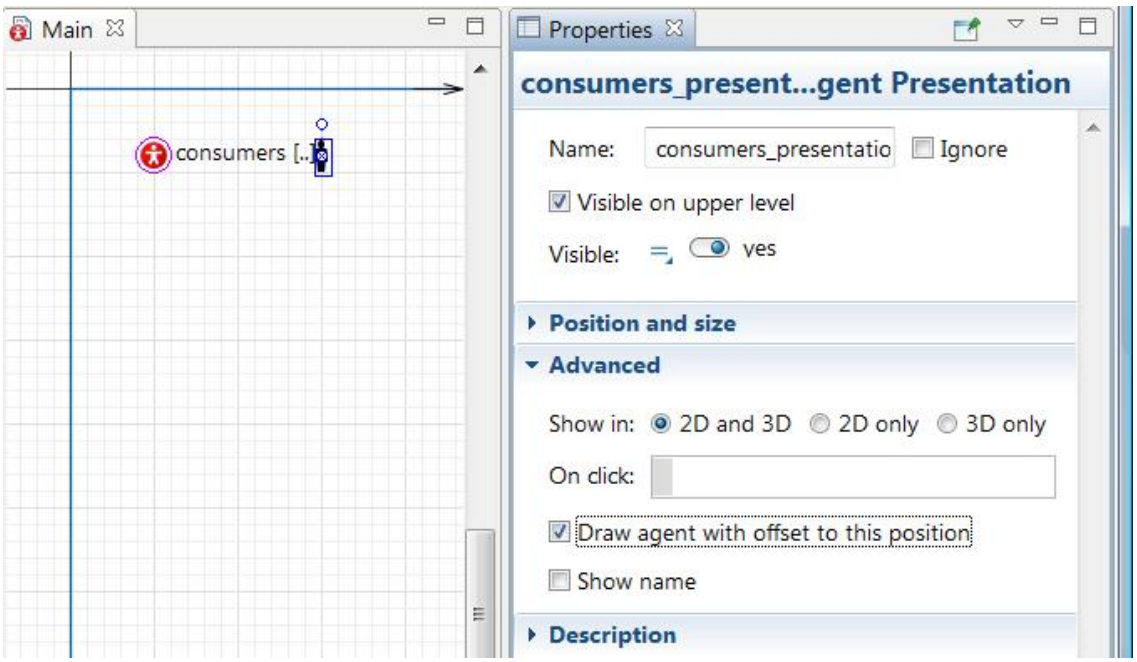


24. Click *Main* in the **Projects** to open its properties in the **Properties** view (you will find **Properties** in the AnyLogic window's right half).

    In the **Space and network** section of *Main* properties, you can adjust the environment settings for the *consumers* agent population.

25. On *Main* diagram, select the agent population's non-editable embedded animation shape , open the **Advanced** properties section, and select the **Draw agent with offset to this position** option.

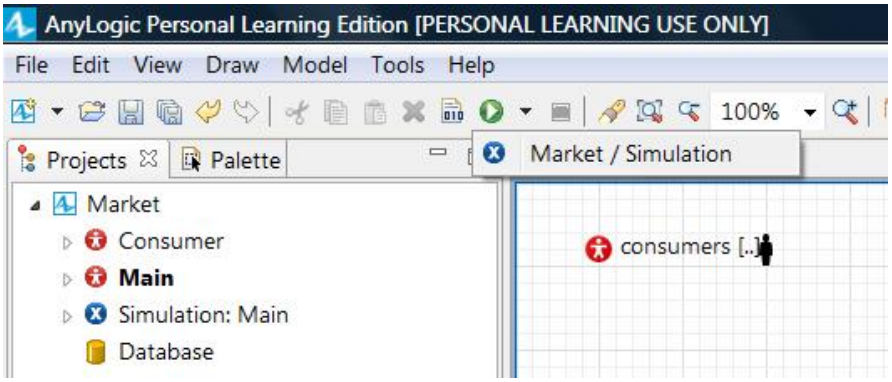
    The animation shape defines the upper-left corner of the 500x500 pixel space where the individual agents will reside when we run the model.



We have finished building this very simple model, and you can now run it and observe its behavior.

26. On the toolbar, click the **Build** button to build the model and check it for errors.

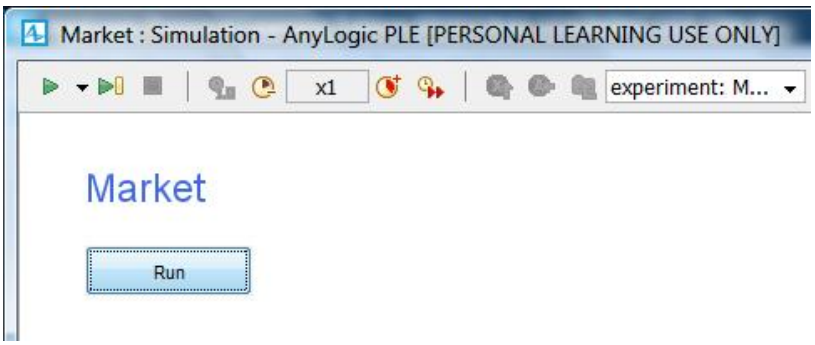
27. Locate the **Run** button, and click the small triangle to the right. Select the experiment you want to run. Choose **Market / Simulation** from the list.



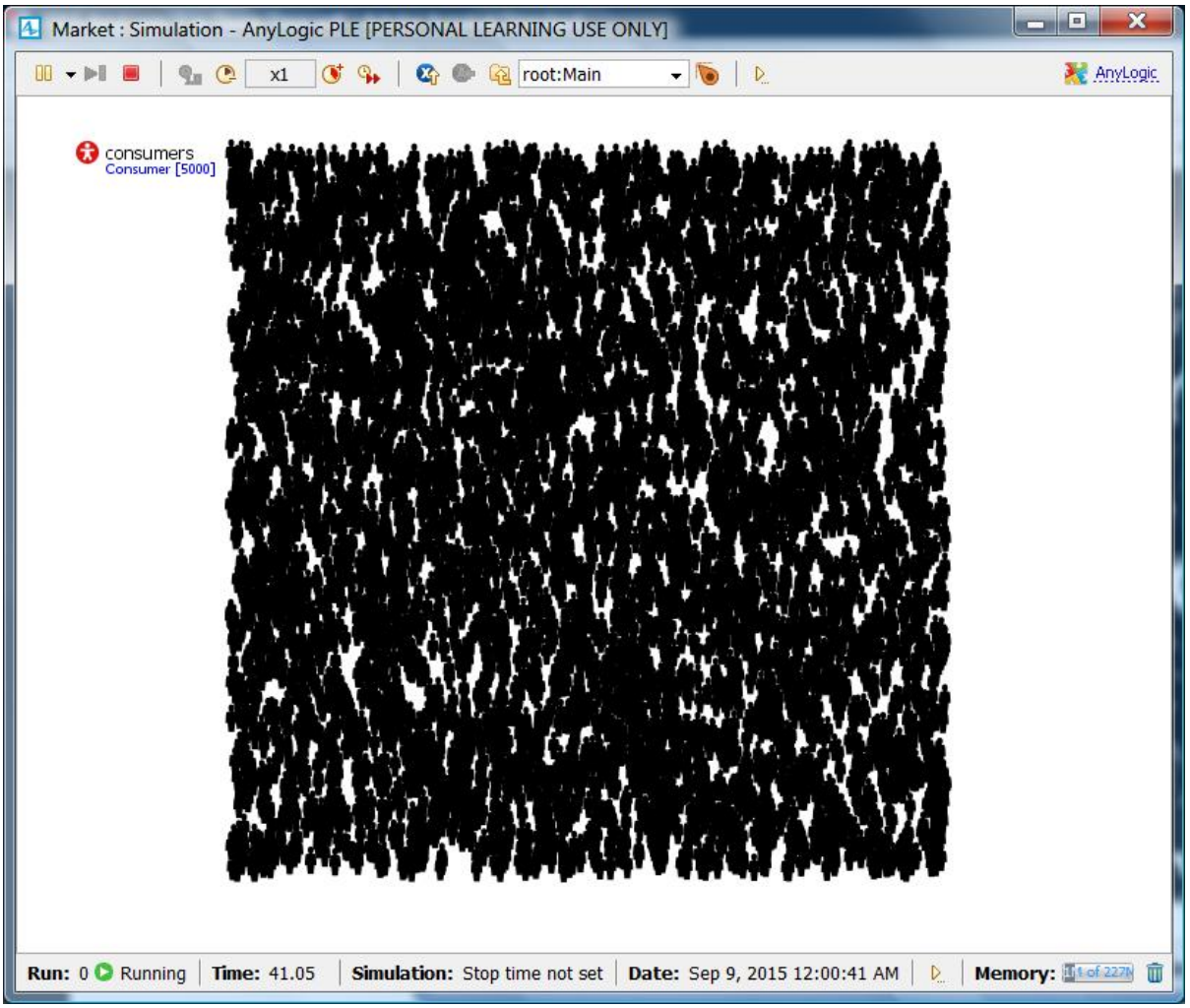
Since you can have several models open at the same time – and each model may have several experiments – you must select the correct experiment.

After you start the model, the presentation window displays the launched experiment *Simulation*. By default, the presentation window displays the model's name and the **Run** button.

28. Click the **Run** button to run the model.



You will see the model's presentation (the presentation you created for *Main* agent) that shows 5000 animations for the agents that comprise the *consumers* population. Since we didn't create any behavior for our agents, the animation appears still.



You can use the toolbar at the top of AnyLogic's presentation window to control the model's execution.

29. We are ready to define the *consumer*'s logic. To continue developing our model, close the presentation window.

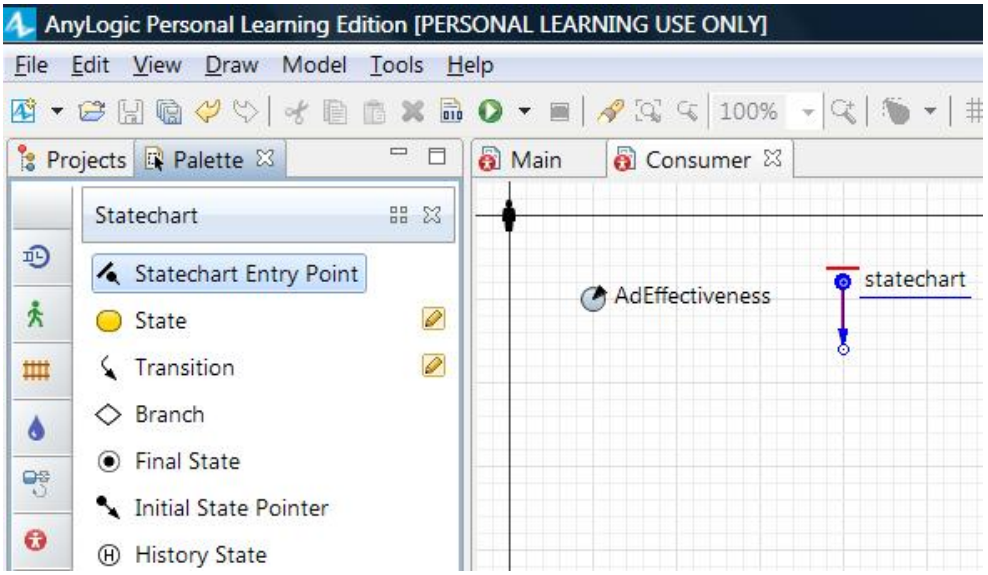
Phase 2. Defining a consumer behavior

We will continue developing our model by defining consumer characteristics and behavior. The best way to define a behavior is to use a statechart.

We will define a consumer's behavior as a two state sequence:

- A consumer in the *PotentialUser* state is only potentially interested in buying the product.
- A consumer in the *User* state has purchased the product.

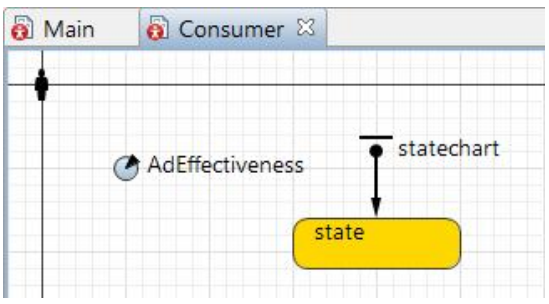
1. In the **Projects** view, open the *Consumer* diagram by double-clicking it. You will see the agent's graphical diagram with the animation figure in the axis origin and the parameter.
2. Start drawing a statechart by drawing two states. Open the **Statechart** palette.
3. Drag the **Statechart Entry Point** from the **Statechart** palette on to the *Consumer* diagram. You start drawing a statechart by adding a statechart entry point. The entry point defines the start of the statechart control flow and the statechart's name.



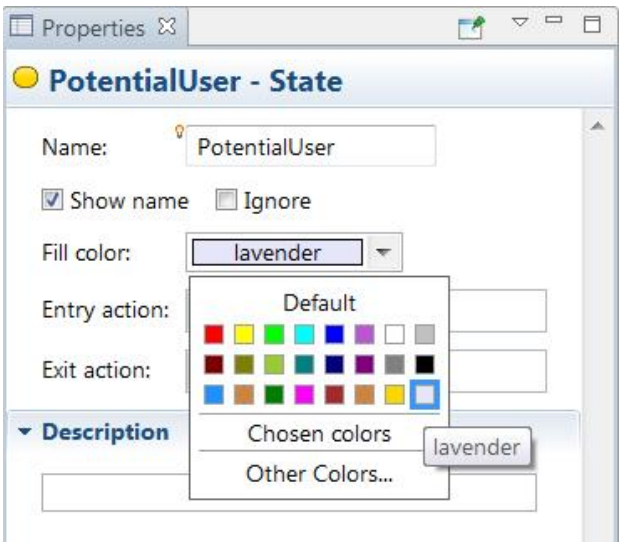
You can see how AnyLogic has highlighted the statechart entry point in red. It means the entry point isn't connected to any state, and the current statechart is invalid.

We add the first state in the consumer's statechart.

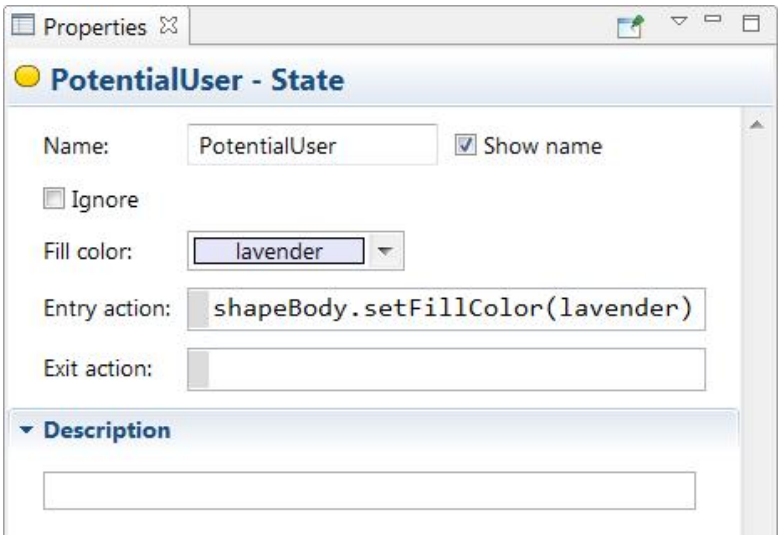
4. Drag the **State** from the **Statechart** palette on to the graphical diagram, and connect it to the statechart entry point.



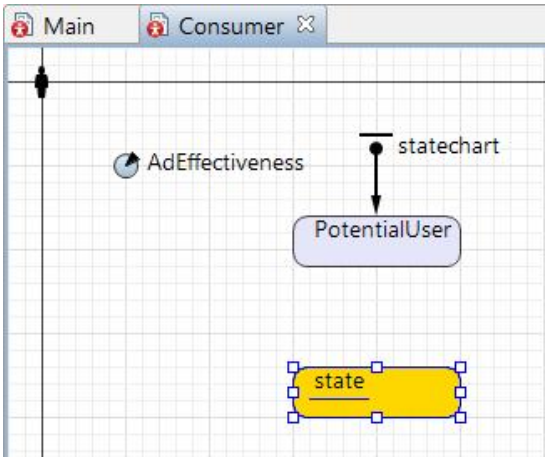
5. Select the state in the graphical editor, and modify its properties. Name the state *PotentialUser*.
6. Use the **Fill color** control to change the state's color to *lavender*.



7. Type the following Java code in the state's **Entry action** field: *shapeBody.setFillColor(lavender).*



8. Add another state in the consumer's statechart:

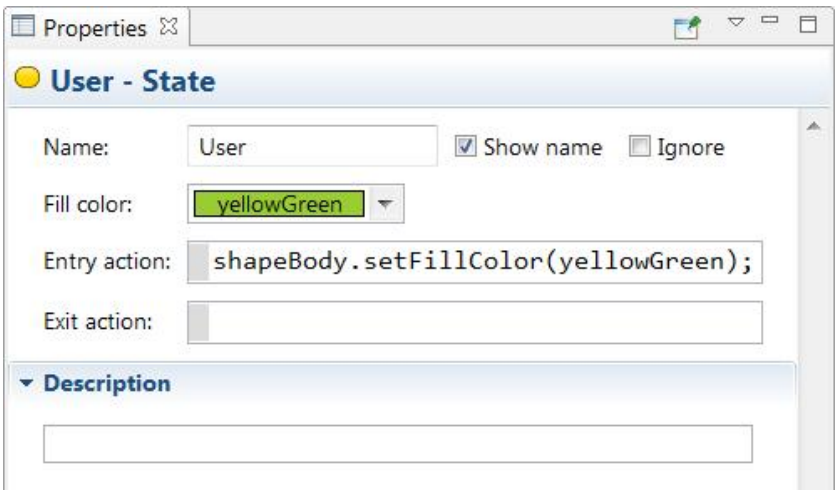


9. Modify the state's properties like you did earlier:

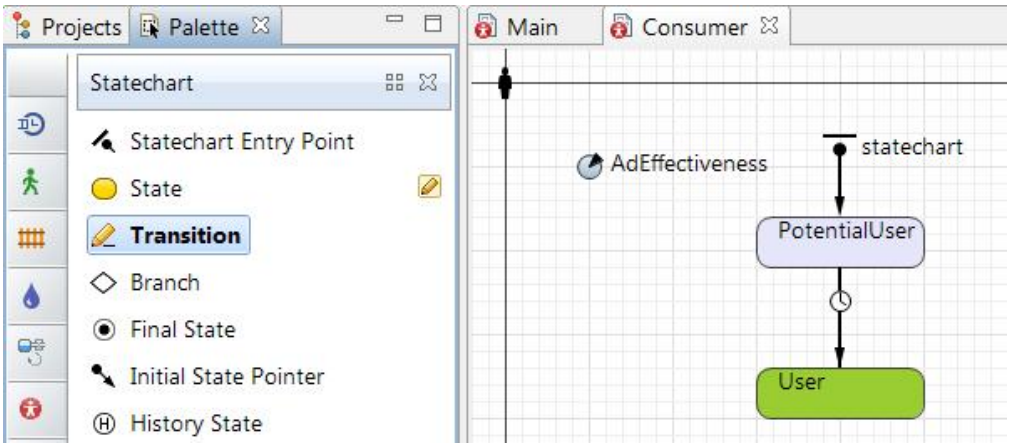
   Name: *User*

   Fill color: *yellowGreen*

   Entry action: *shapeBody.setFillColor(yellowGreen);*



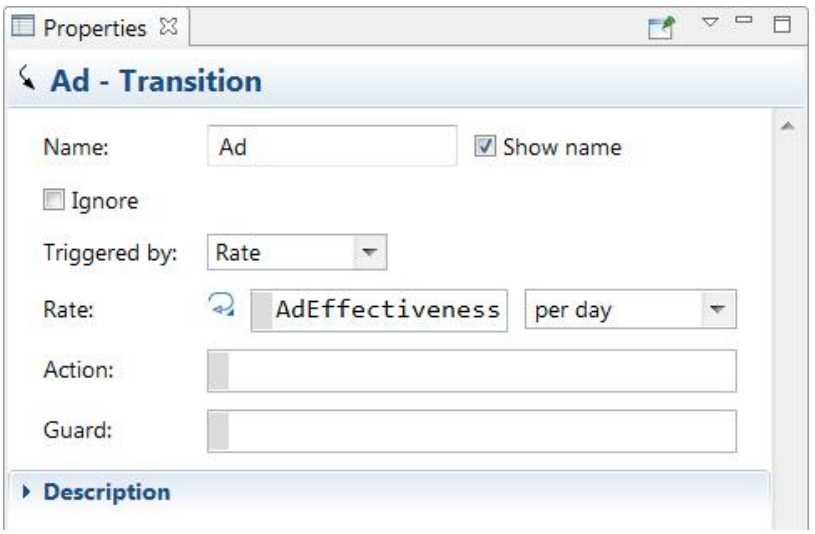
10. Draw a transition from *PotentialUser* to *User* state to model how persons purchase the product and become product users. To do so, double-click the **Statechart** palette's **Transition** element (the element's palette icon should change to), click *PotentialUser* state, and click *User*.



11. Name the transition *Ad* to represent "advertising".

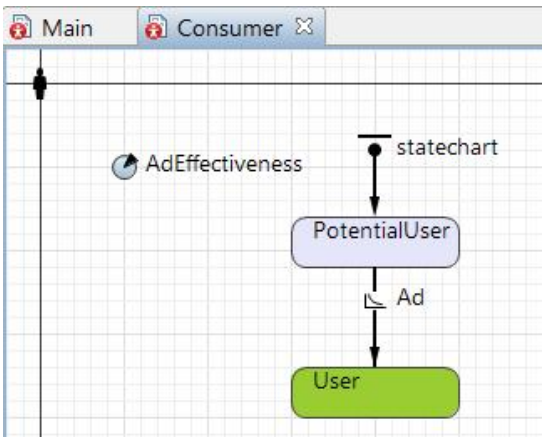
12. Select the **Show name** checkbox to display the transition's name on the graphical diagram.

13. The transition from *PotentialUser* to *User* state will model how advertising leads the person to buy the product. In the **Triggered by** list, click **Rate**. In the **Rate** field, type *AdEffectiveness*, and then click **per day**.



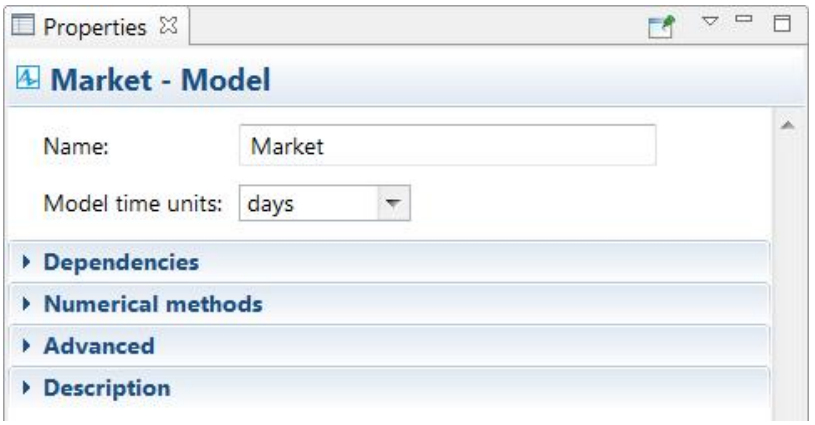
You can see that the icon drawn over the transition has changed from to . This sign shows the transition's *trigger type*.

To move the transition's name or icon, select the transition, and use your mouse to drag the corresponding element to a new location.



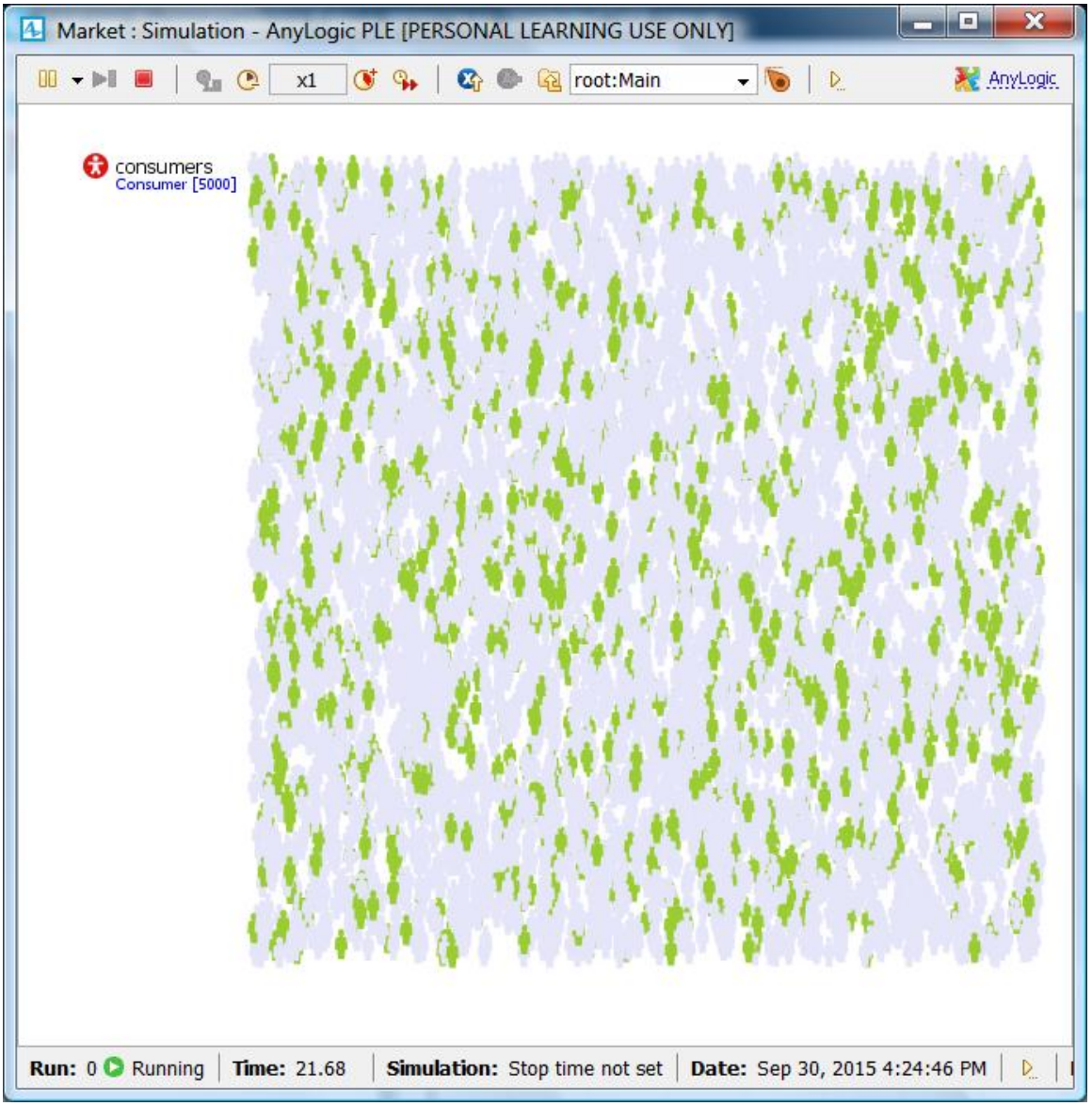
Our transition is triggered with the specified rate. In our case, when the statechart enters the state *PotentialUser*, a draw from the exponential distribution is made and the timeout is set up. Each consumer's adoption time will differ, though an average 1% of potential users will buy the product on a given day.

14. Now, let's set up the model's time units. To tune the model setting, switch from **Palette** to **Projects**, and then click the model item in the tree (the tree's top object, *Market*). In the **Properties** view, choose *days* as the **Model time units**.



15. Run the model. The population should gradually turn green – a change that represents the effect of advertising - until every consumer buys the product.

    When advertising's effects cause an agent to purchase the product, the agent's state *User* becomes active, the state's **Entry action** is executed, and the agent animation shape's color changes to *yellowGreen*. As more people purchase the product, you will see your model's agent animations gradually turn green.



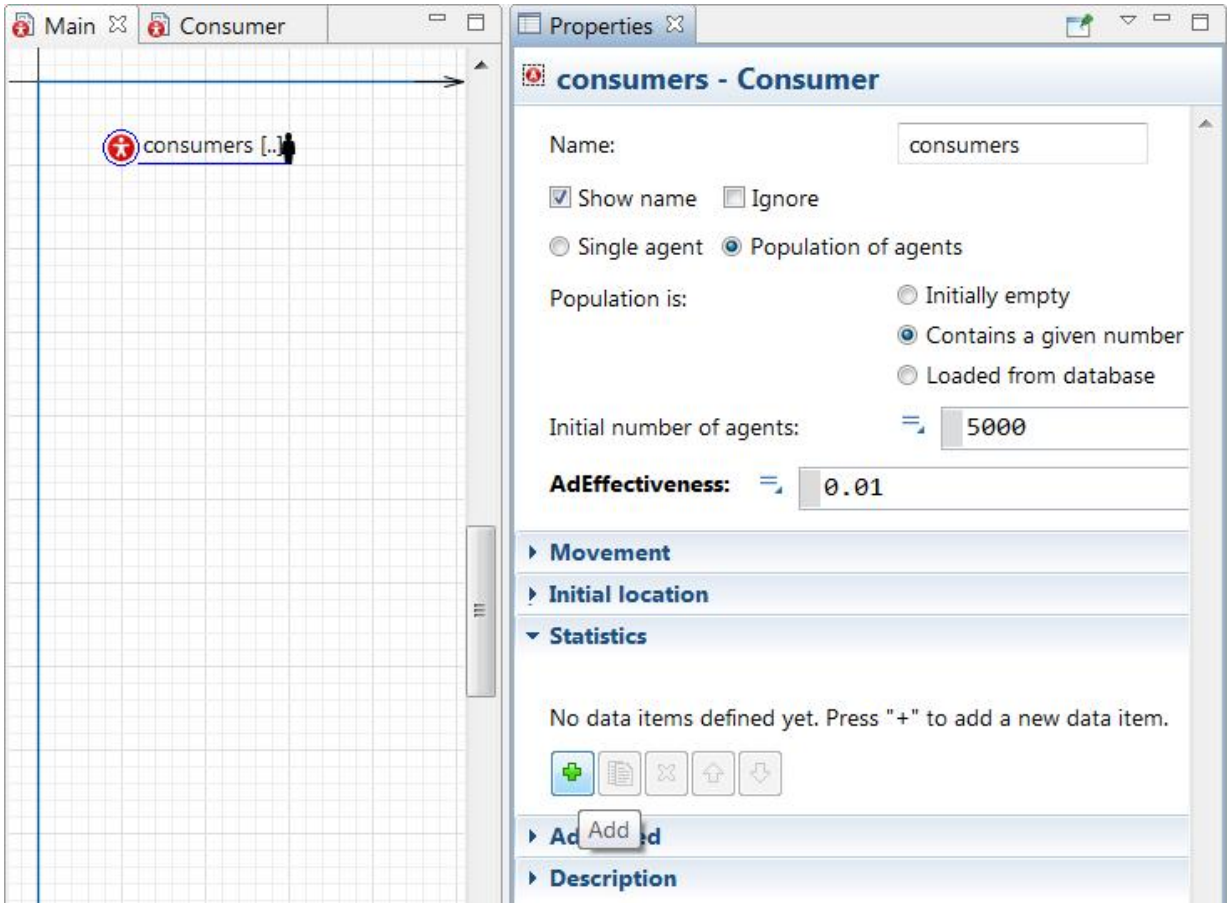
16. To adjust the model's execution speed, click the toolbar's **Slow down** or **Speed up** buttons. If you increase the speed to *10x* – you will see the speed at which the population turns green also increase.

Phase 3. Adding a chart to visualize the model output

We want to know how many people have purchased our product at a given moment. With that in mind, we will define functions that count our product's users and potential users, and then add a chart to show the dynamics.

1. First, define a function to count potential users. To add a new function that collects statistics for agents, open the diagram of the agent type *Main*, select the agent population *consumers*, and go to the **Statistics** properties section.

2. Click the  **Add statistics** button.

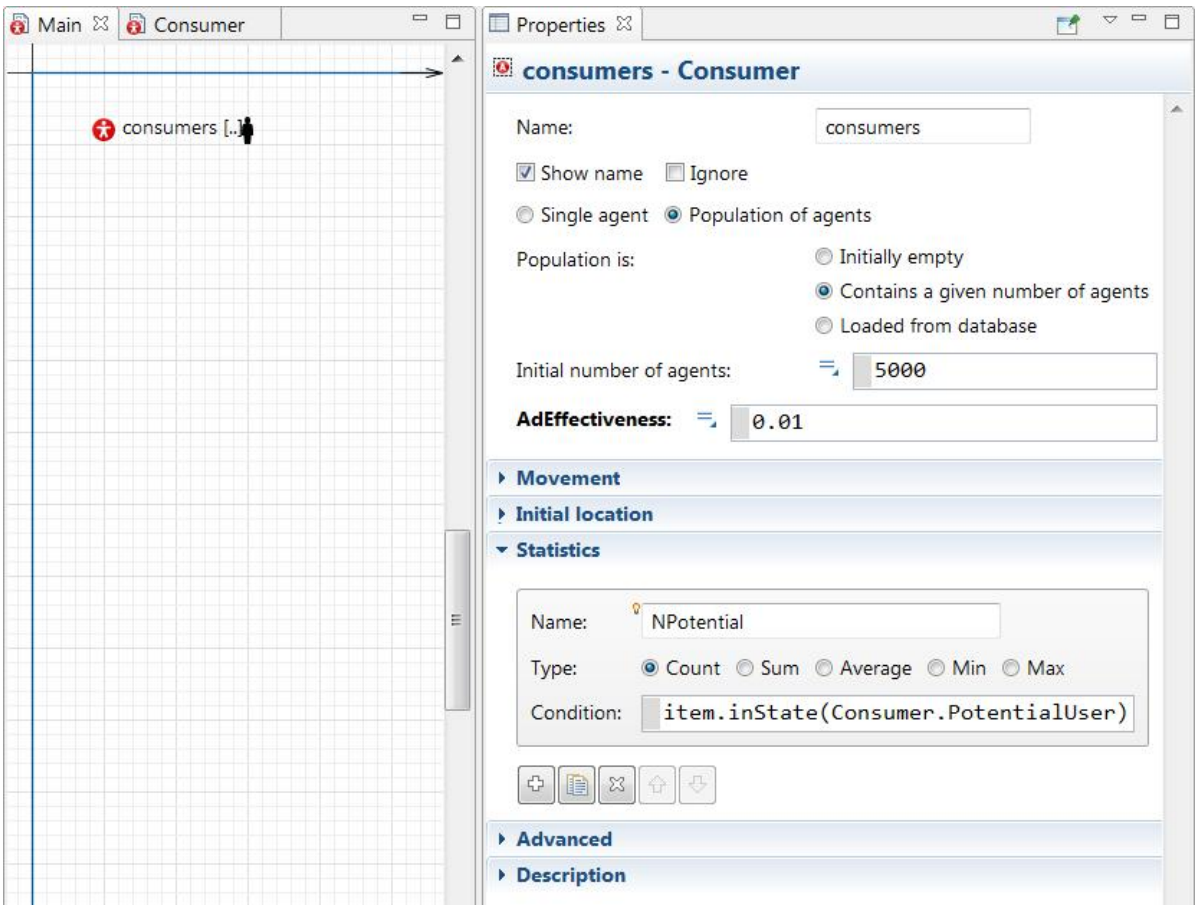


We need to determine how many agents are in the *PotentialUser* state.

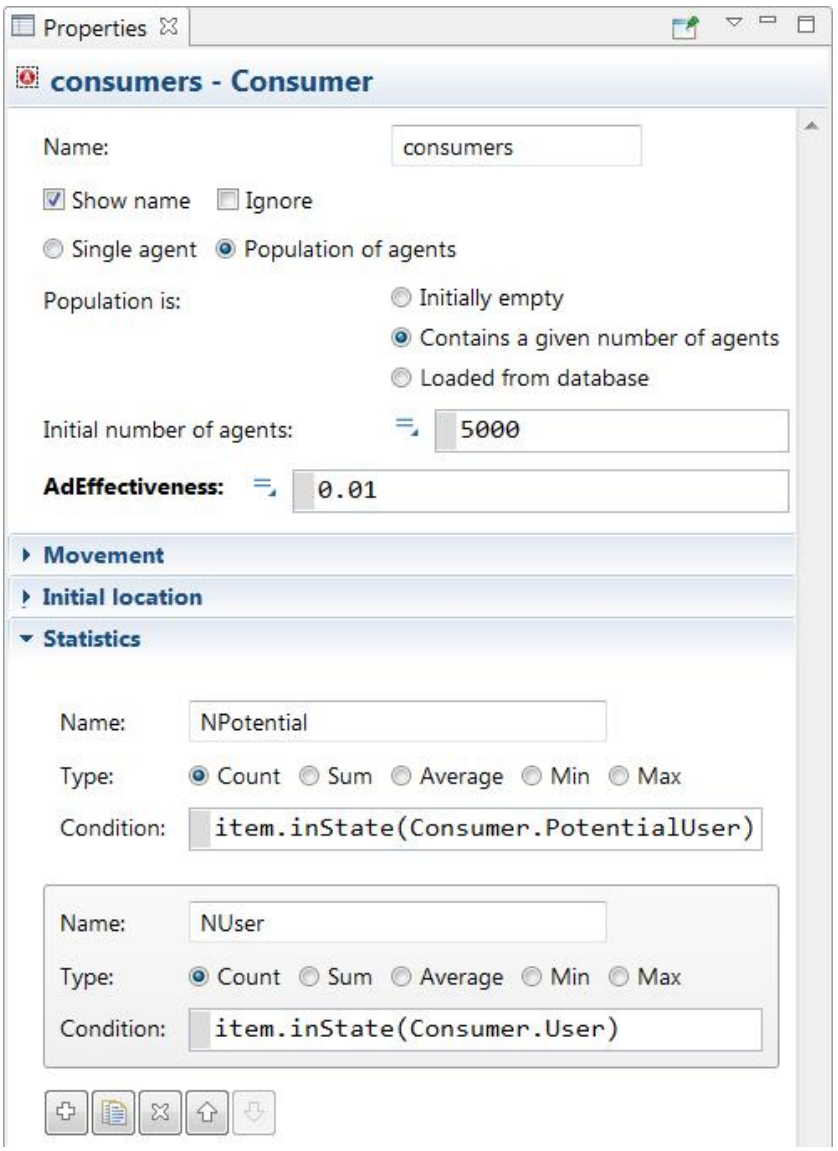
3. Define the function of type **Count** with the **Name** *NPotential*. The statistics of type **count** iterates through a given population – in our case, the number of agents – to count those that meet the selected condition.

4. Enter *item.inState(Consumer.PotentialUser)* as the function **Condition**.

   - *item* represents the agent being currently checked in the iteration.
   - *inState()* is a function that checks whether the specified state of the statechart is active.
   - *PotentialUser* is the name of the agent-defined state, which is why it needs the agent type prefix *Consumer*.

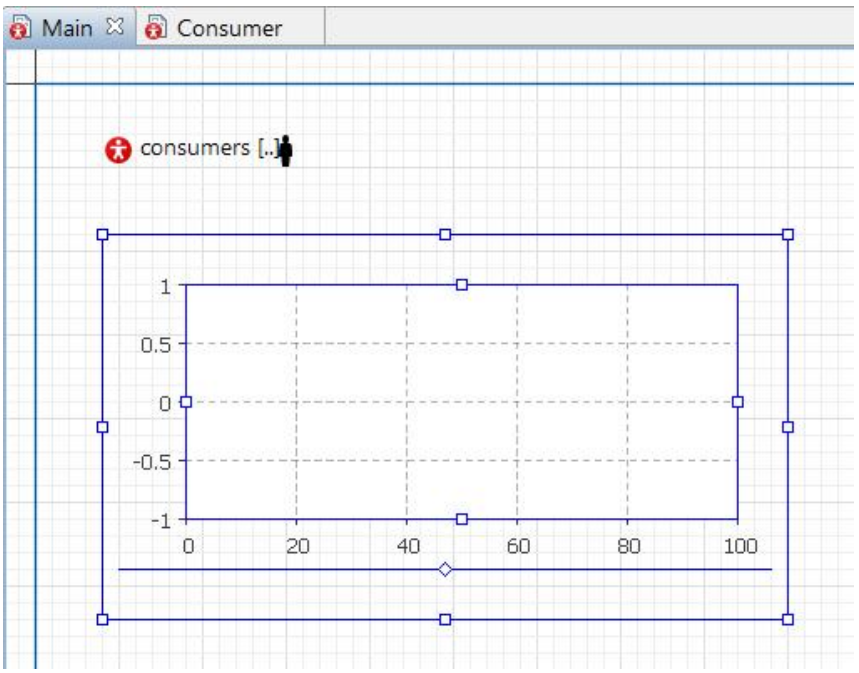


5. Define a second statistics function to calculate the number of product users. Name it *NUser* and let it count the number of agents, conforming the **Condition** *item.inState (Consumer.User)*. You can duplicate the other statistics function by clicking the  **Duplicate** button and changing its **Name** and the **Condition**.



Now, let's add a chart to show the statistics these functions collect and display the adoption process dynamics.

6. Open the **Analysis** palette, and drag the **Time Stack Chart** from the **Analysis** palette on to the Main diagram to create a chart that will display the dynamics of users and potential users. Increase the time stack chart as shown in the figure below:



Add two data items for the chart to display. Here we will call our statistics functions *NUser* and *NPotential* we have defined for *consumers* population on the previous step.

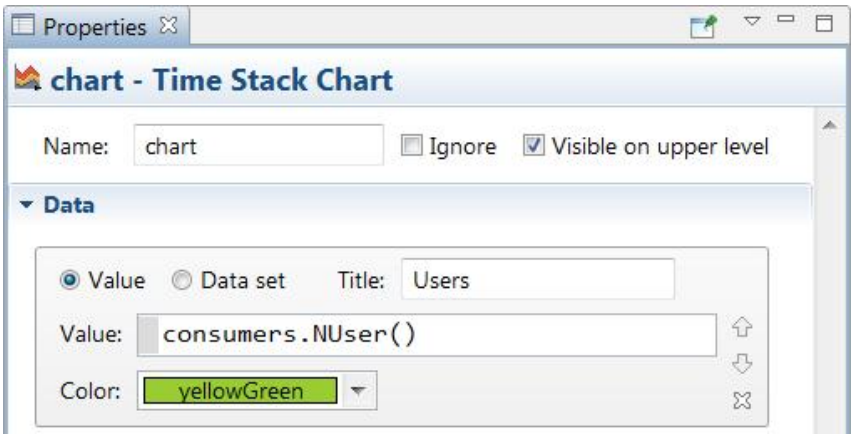
7. Click **Add data item** to add the statistics you want to draw on the chart.



8. Modify the data item's properties:

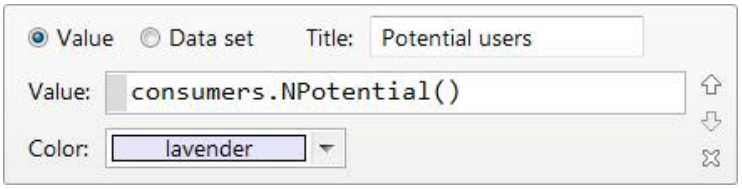
   - **Title**: *Users* – the data item's title.
   - **Color**: *yellowGreen*
   - **Value**: *consumers.NUser()*

   Our agent population name is *consumers*, and *NUser()* is the statistics function that we defined for this population.



9. Add one more data item:

   - **Title**: *Potential users*
   - **Color**: *lavender*
   - **Value**: *consumers.NPotential()*



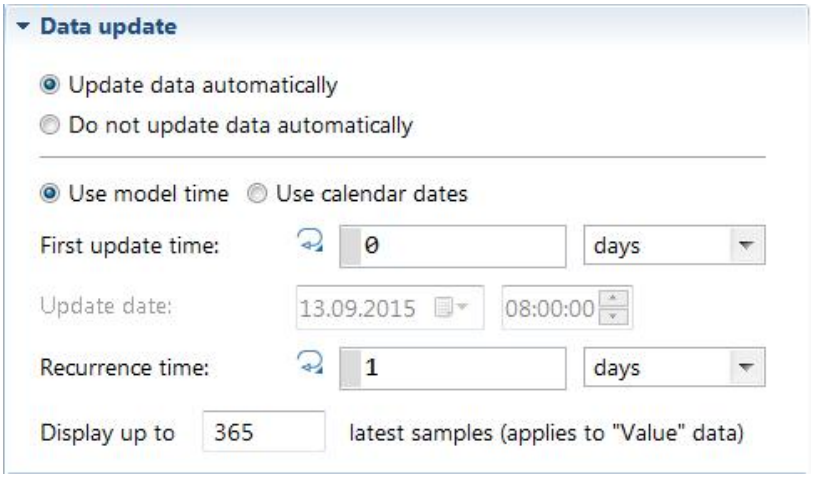
   Since we want to show a one year range, we need to adjust the chart's settings.

10. Go to the **Scale** section and set **Time window** equal to *1 years*.

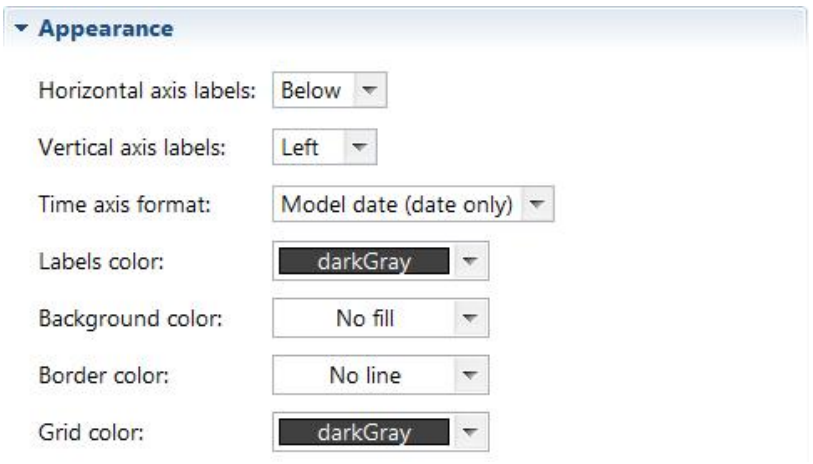
11. Since our chart will show statistics for *consumers* population and our model has 5000 consumers, set the chart's **Vertical** scale to **Fixed**, and enter *5000* in the **To:** box.



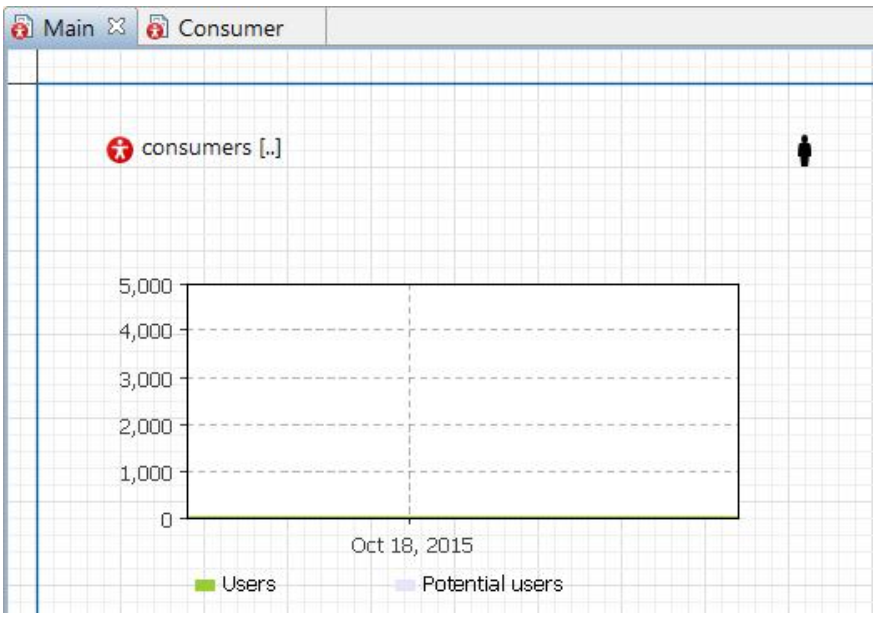
12. Now that we have set the time window, change the maximum number of data samples that the chart displays by navigating to the section **Data update** and setting **Display up to** *365* **latest samples**. Since we will add one data sample each day, 365 data samples is an ideal amount for a one year range.



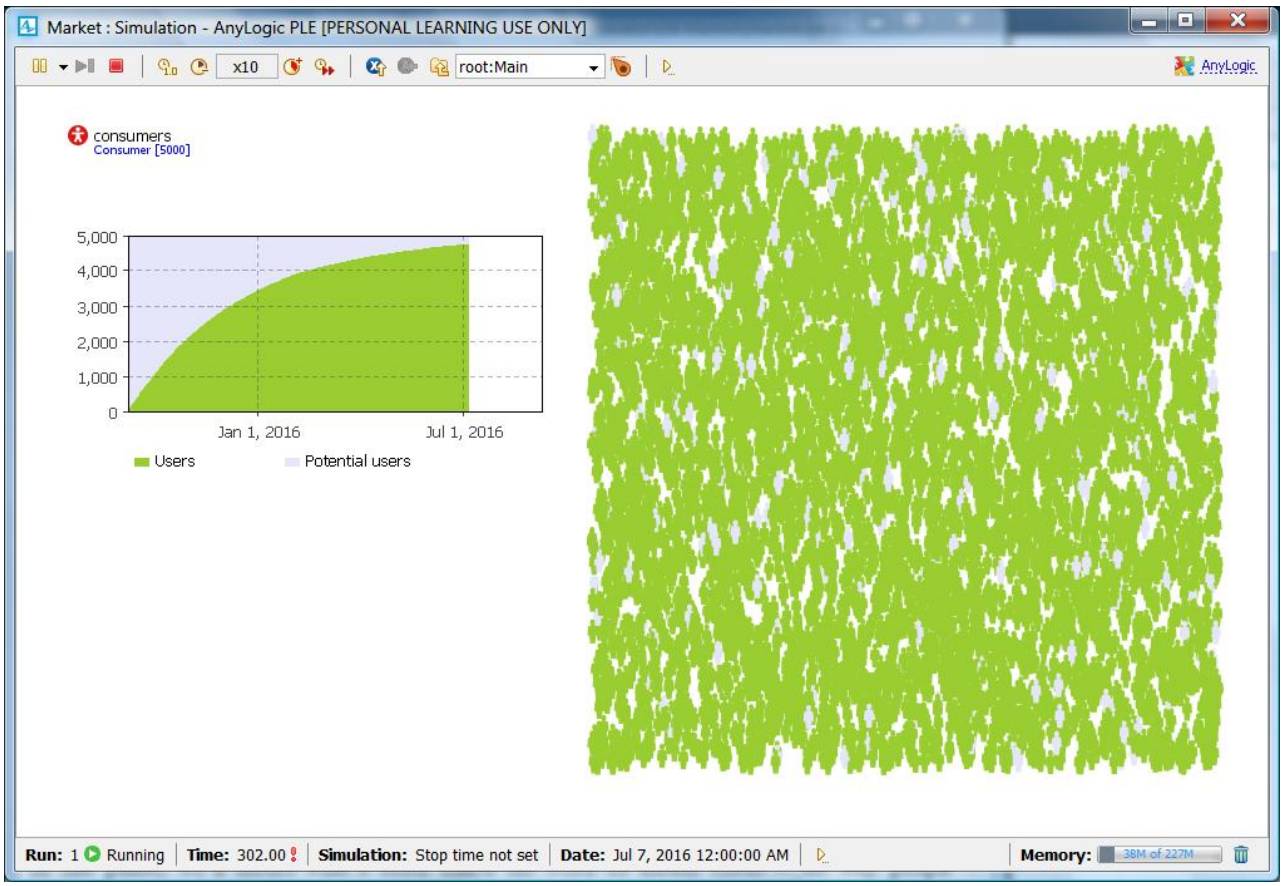
13. Go to the time stack chart's **Appearance** properties and set it to display **Model date (date only)** near the time axis.



14. On the *Main* diagram, move the presentation of the *consumers* agent population to the right.



15. Run the model and use the time stack chart to review the process.



Phase 4. Adding word of mouth effect

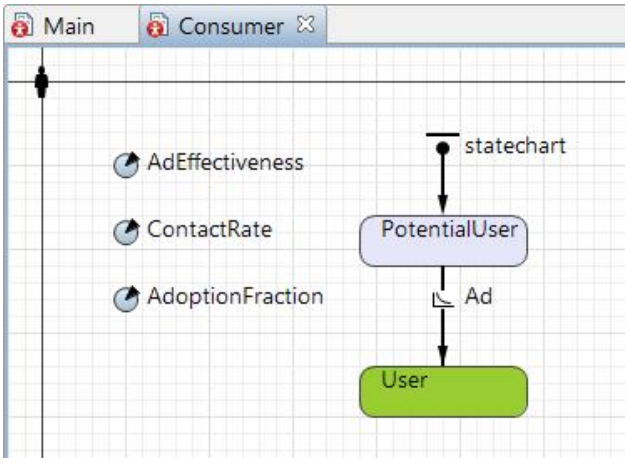
In this phase, we will model what's often called the word of mouth effect – the way people persuade others to purchase our product.

- Allow people to contact one another. In our model, a consumer contacts an average of one other person each day.
- Our product's current users may influence potential users during these meetings. We will define the probability of a potential user buying the product as *AdoptionFraction = 0.01*.

Let us develop the model's logic by adding two consumer parameters: *ContactRate* and *AdoptionFraction*.

1. In the **Projects** tree, open *Consumer* diagram by double-clicking *Consumer*.
2. Add a parameter to define a consumer's average daily contacts. Drag the **Parameter** from the **Agent** palette on to the diagram.
3. Name the parameter *ContactRate*.
4. The rate is 1 contact per day, so type *1* as the parameter's **Default value**.
5. Add another parameter - *AdoptionFraction* - to define a person's influence on others, a number that we will express as the percentage of people who will use the product after they come into contact with the consumer. Leave the default parameter's **Type:** *double*, and set the **Default value:** *0.01*.

   The *Consumer* diagram should look like this:

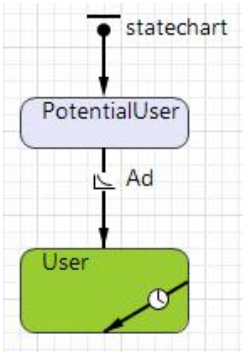


Now, we will allow our agents to interact. This represents the word of mouth discussions that will convince a percentage of consumers to buy the product.

In our model, only users who are in the *User* state will send messages. The best way to define an activity that an agent performs while in a state – in other words, an activity they perform without exiting their current state – is to use an *internal transition*.

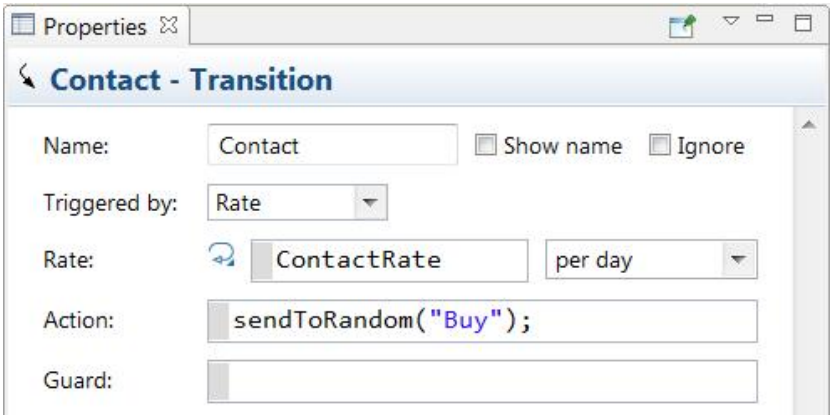
6. Open the *Consumer* diagram, and increase the *User* state to fit the internal transition we will draw inside the state on the next step.

7. Draw an internal transition inside the *User* state. To draw a transition like the one shown below, drag the **Transition** from the **Statechart** palette inside the state so the transition's start point lies on the state border. Afterward, you can move the transition end point to another point on the state border. To add a salient point, double-click the transition.



8. Modify the transition properties. This transition will occur with the specified **Rate** *ContactRate* (use code completion rather than typing the parameter's full name). Name the transition *Contact* and set it to show its name.

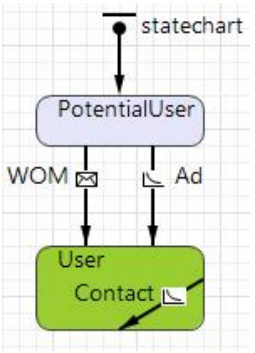
9. Specify the **Action** that will be executed on triggering this transition (use the code completion to write the code): *sendToRandom("Buy");*



Since we want our product's users to speak to potential users, we will set up a cyclic transition in the state *User*. Each time the transition takes place, the code *sendToRandom("Buy");* causes the consumer to randomly choose another agent and send them a *"Buy"* text message. If the agent who receives the message is a potential user (in other words, if the receiving agent is in the state *PotentialUser*), the receiving agent's state will change to *User*.

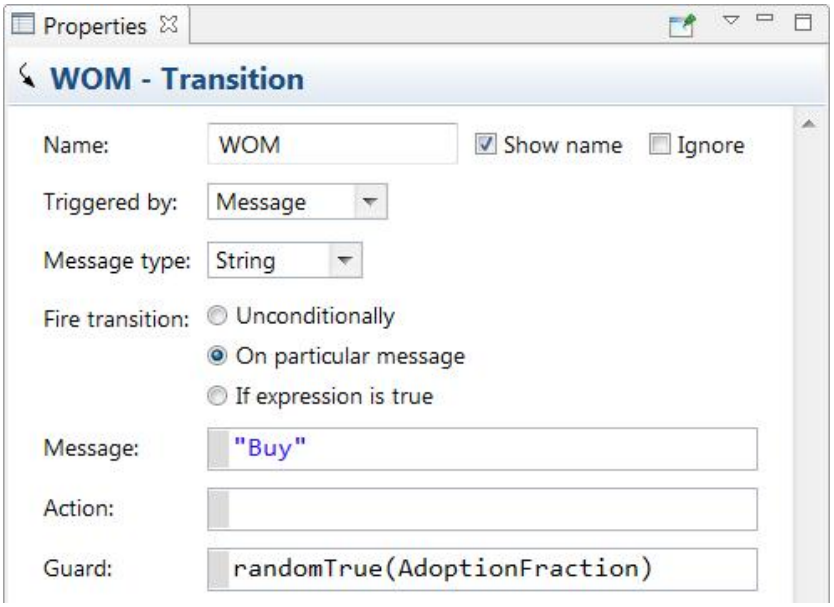
Let's add this transition now:

10. Draw another transition from *PotentialUser* to *User* state, and name it *WOM*. This transition will model purchases caused by word of mouth.



11. Modify the transition properties:

    - In the **Triggered by** list, click **Message**.
    - In the **Fire transition** area, select **On particular message**.
    - In the **Message** field, type *"Buy"*
    - Since we know not every contact is successful – in other words, a contact may not convince the potential user to buy our product – we will use *AdoptionFraction* to make successful contacts less common. Specify the transition's **Guard**: *randomTrue(AdoptionFraction)*

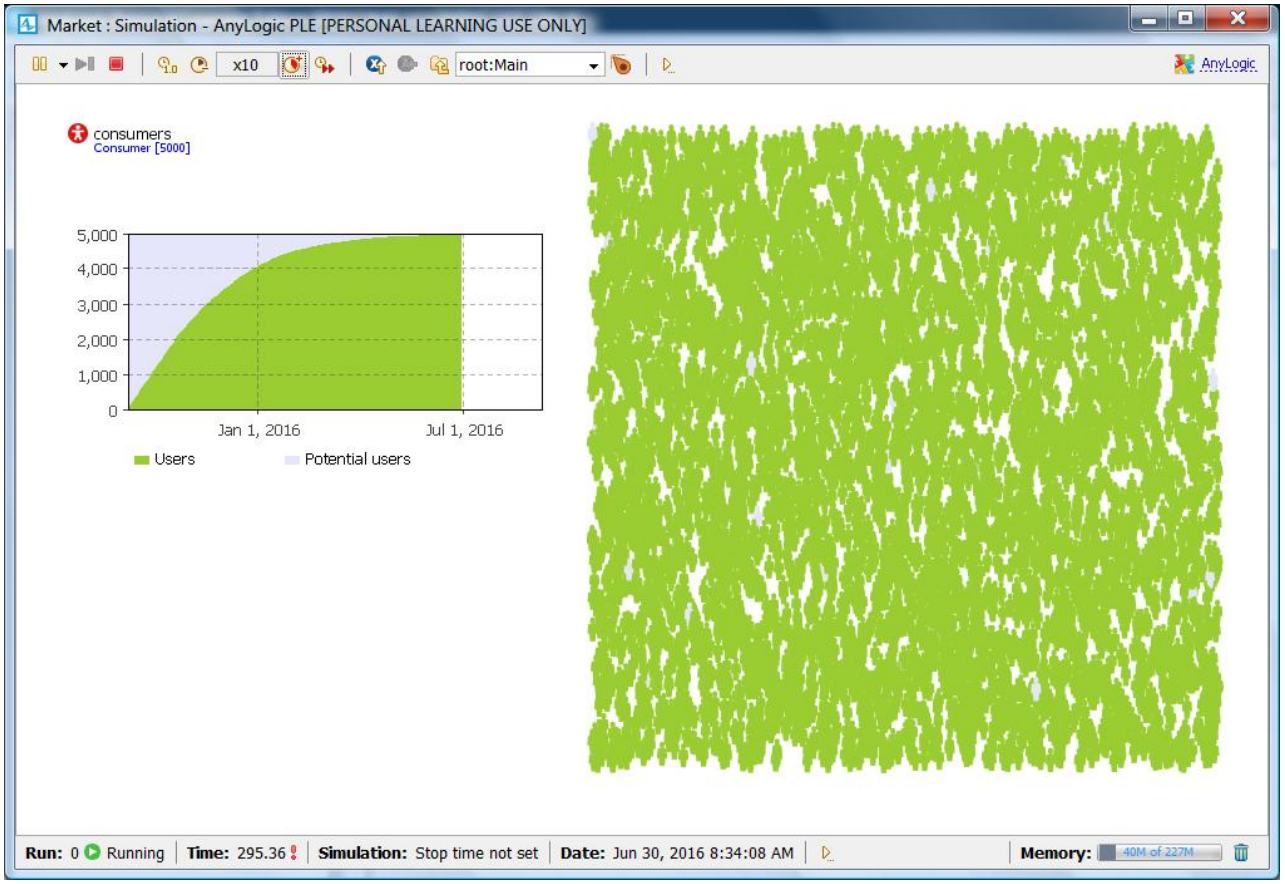


This is the last step in modeling word of mouth marketing. AnyLogic forwards the message from another agent to the statechart, and, if the statechart is in the state *PotentialUser*, it causes an immediate transition to the *User* state. If the statechart is in any other state, it will ignore the message.

12. In the **Projects** view, you may see an asterisk near the model item that shows your model has unsaved changes. On the toolbar, click **Save** to save your model.

13. Run the model.

    The market saturation should occur more quickly, and the chart shows the well-known S-shaped product adoption curve.

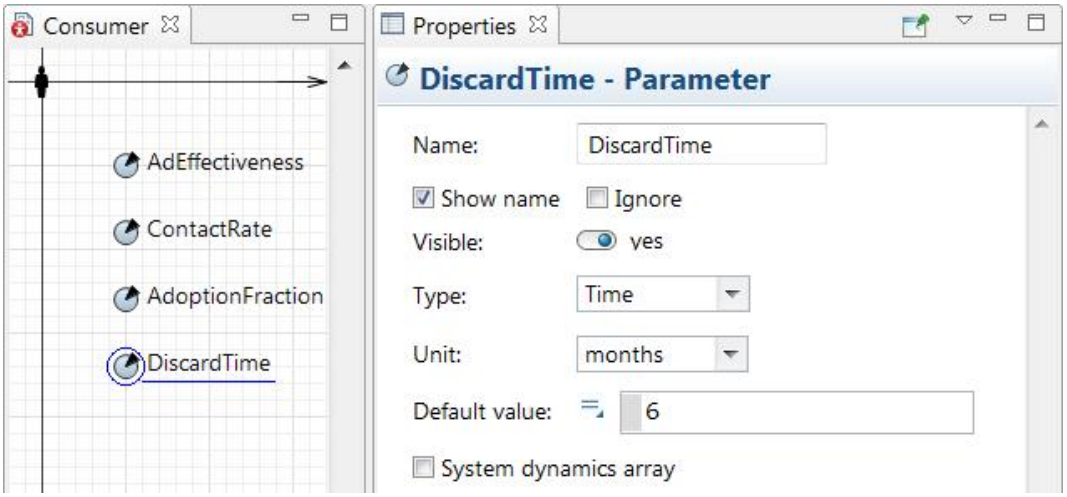


Phase 5. Considering product discards

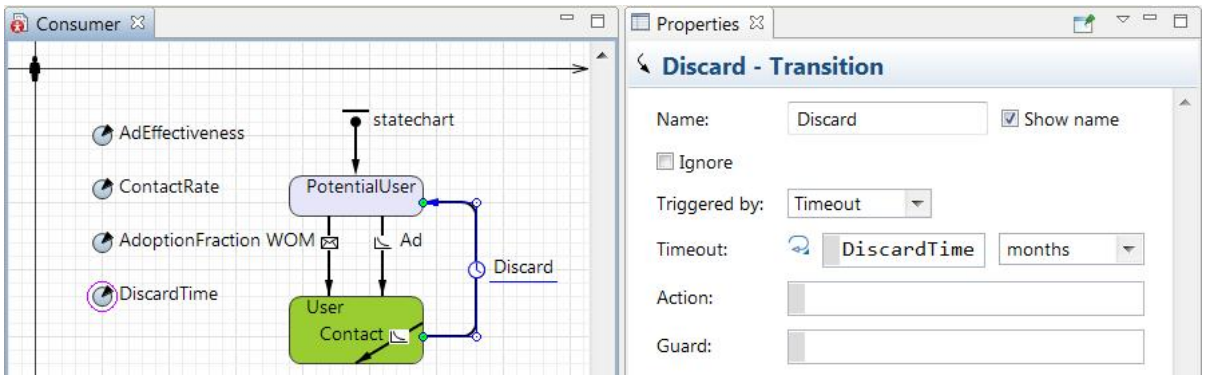
In this phase, we will model product discards.

- Let us assume the average duration of our product's active use is six months.
- Once a user discards or consumes the product, they will need a replacement. We will model repeat purchase behavior by assuming adopters become potential adopters when they discard or consume their first units (in other words, when the *User* reverts to the *PotentialUser* state).

1. Open the *Consumer* diagram and add a *DiscardTime* parameter.
2. This parameter will define our product's lifespan. Choose **Time** as the parameter's **Type**, click **months** in the **Unit** list, and type *6* as the **Default value**.

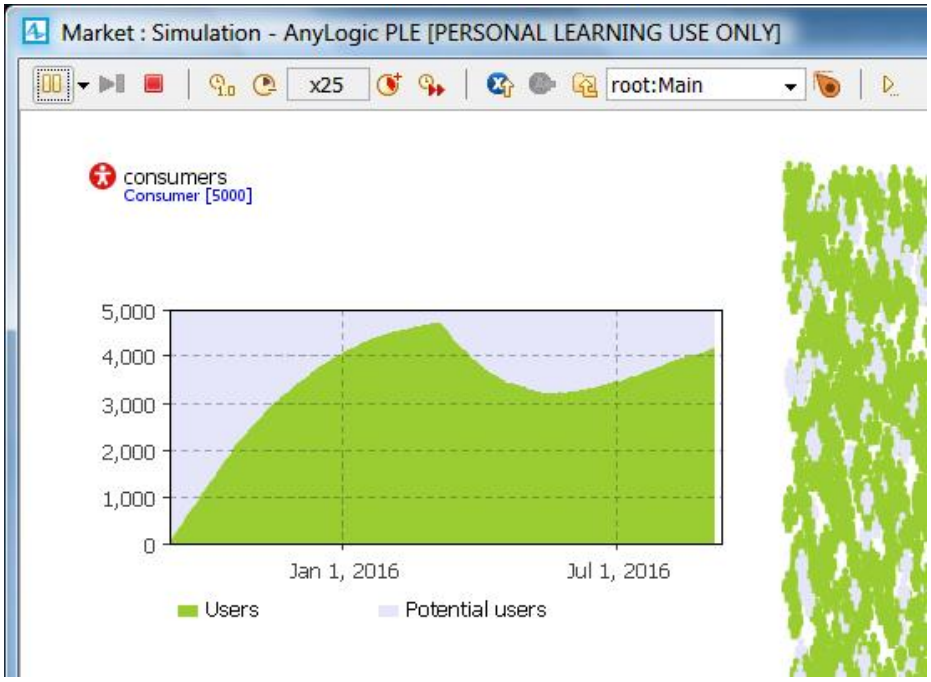


3. Draw a transition from *User* to *PotentialUser* state to model product discards. To draw a transition with salient points like those shown in the figure, double-click the **Transition** element in the **Statechart** palette (this should change the element's icon in the palette to), click the transition's source state *User*, click at the salient point places, and click the target state *PotentialUser*.
4. Name the transition *Discard* and set it to be triggered by a constant timeout *DiscardTime*. In the list to the right, click **months**.



Our work to model product discards is complete, and any discards will generate an immediate need to purchase a replacement.

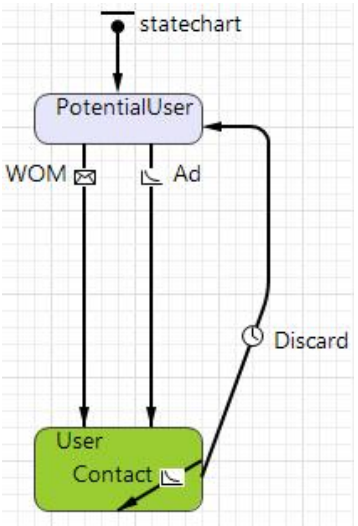
5. Run the model and watch how discards affect adoption dynamics. Even after our product saturates the market, you will notice occasional product discards.



Phase 6. Considering delivery time

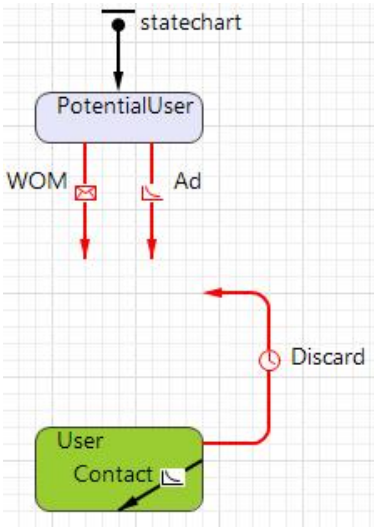
Our model assumes the product is always available and the transition from *PotentialUser* to *User* is unconditional and immediate. Now, we will improve the model by adding a state to the statechart that reflects the amount of time between an agent's decision to purchase the product and the time they receive it.

1. Prepare a place for another state between *PotentialUser* and *User* by moving the *User* state toward the bottom of the screen.

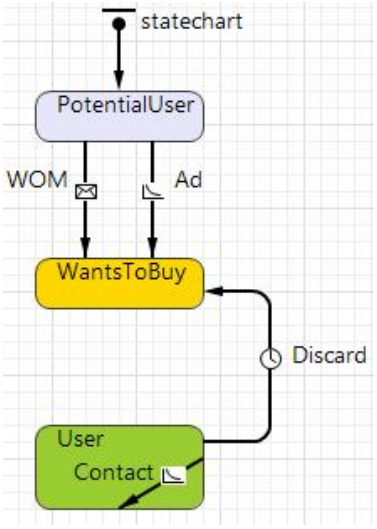


2. Disconnect the *User* state from the transitions.

   Select the *WOM* and *Ad* transitions, move their end points toward the top of the screen, and disconnect the *Discard* transition from *PotentialUser*. Afterward, you will notice the disconnected transitions are drawn in red.



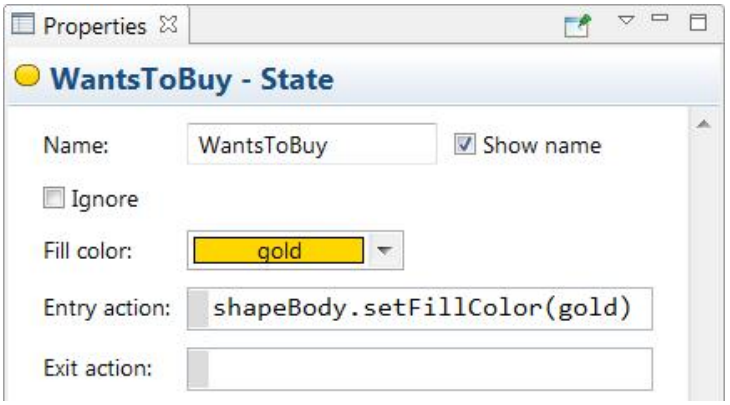
3. Add another **State** from the **Statechart** palette to the middle of the consumer's statechart and name it *WantsToBuy*. Consumers in this state have decided to purchase the product, but they have not done so.



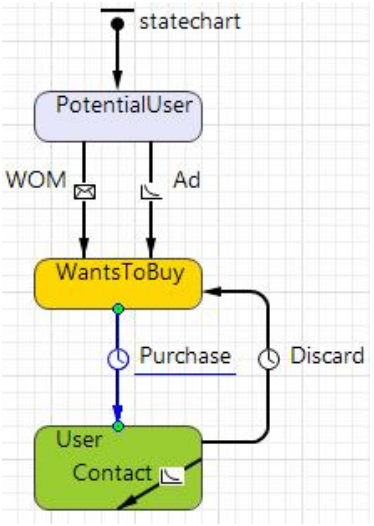
4. Reconnect transitions to the middle state: the *WOM*, *Ad*, and *Discard* transitions should now end in the *WantsToBuy* state.

5. Modify *WantsToBuy* similar to other states:

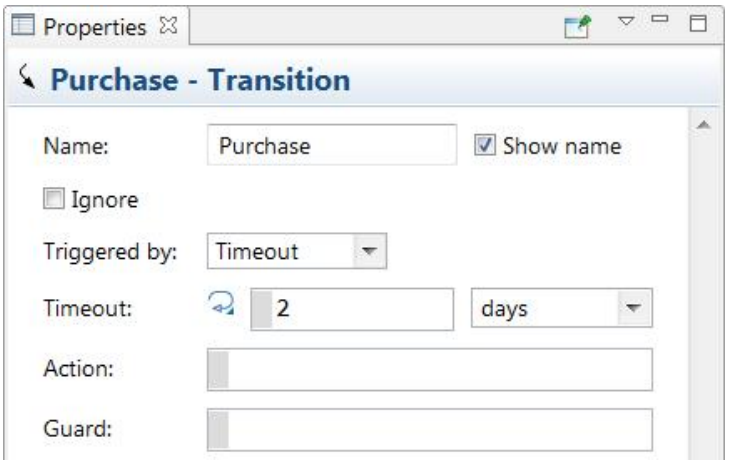
   - **Fill color**: *gold*
   - **Entry action**: *shapeBody.setFillColor(gold)*



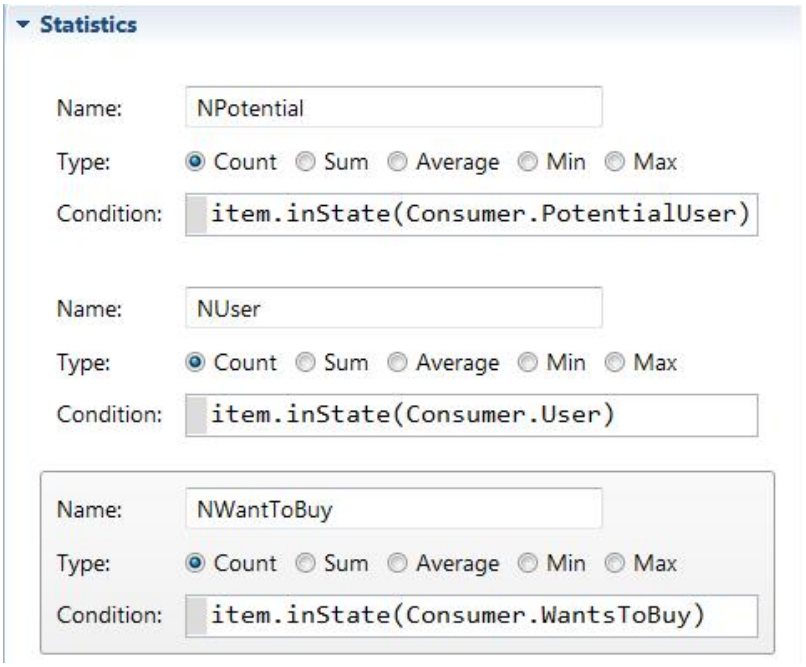
6. Add a transition from *WantsToBuy* to *User* state to model the product shipment and name it *Purchase*.



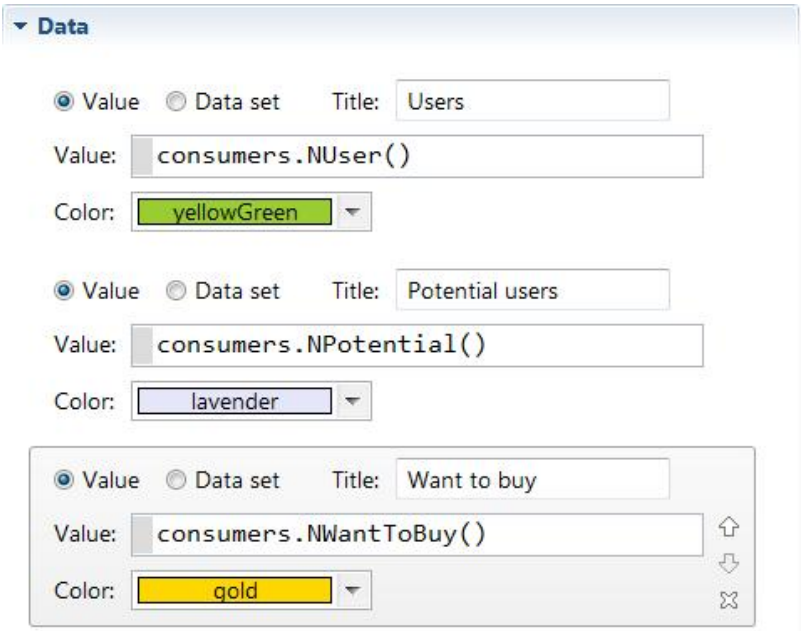
7. Let us assume it typically takes a user two days to get the product. This means once the consumer's statechart enters the state *WantsToBuy*, it will proceed to the state *User* with a two-day delay. With this in mind, set 2 days timeout for the *Purchase* transition:



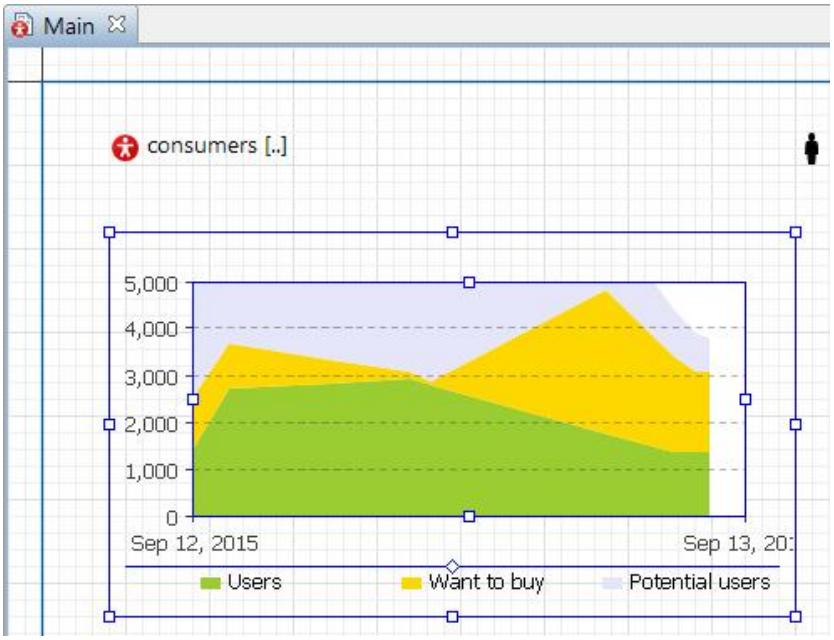
8. Define one more statistics function to count the product's market-driven demand. In the editor of *Main*, click the *consumers*, go to the **Statistics** properties section, and add a statistics item: *NWantToBuy* with condition *item.inState(Consumer.WantsToBuy)*.



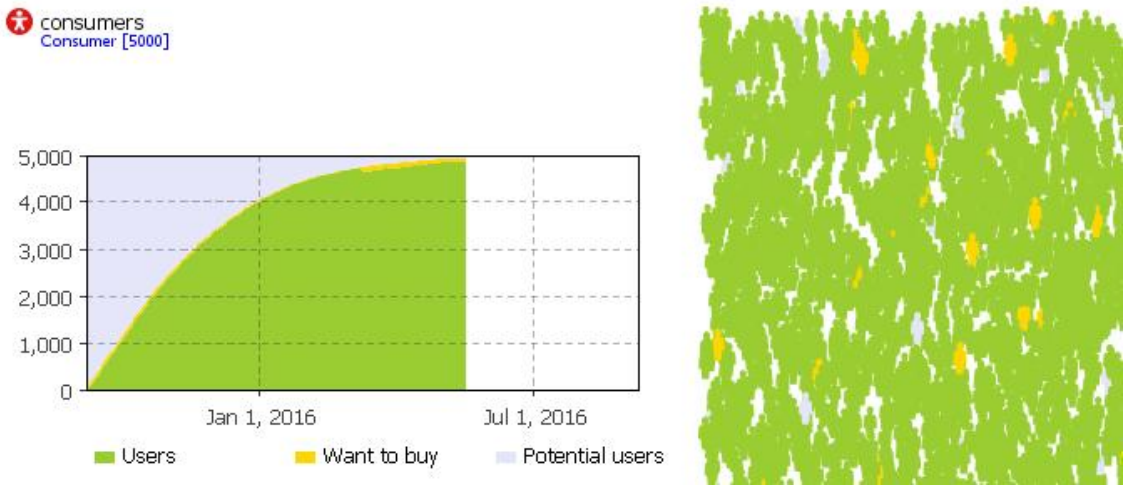
9. On *Main*, select the time stack chart, and add another data item to be displayed with the chart: *consumers.NWantToBuy()* with the title *Want to buy* and color *gold*.



10. Make the newly-defined data item second in the list by selecting the item's section and clicking the "up" button ⇧.



11. Run the model, and you will notice AnyLogic displays the number of consumers who are waiting for the product in yellow.



Phase 7. Simulating consumer impatience

Our model needs to address the varying amounts of time that consumers are willing to wait for their product's delivery. If the delivery time exceeds the time a consumer is willing to wait, the consumer will reconsider their decision and return to being a potential user rather than one who wants to buy.

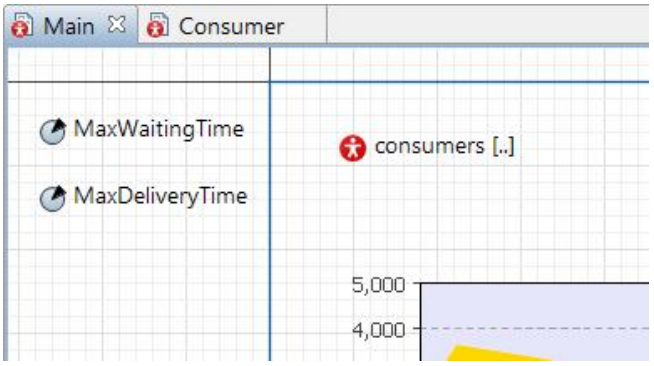
Let us start by defining two parameters in *Main*: maximum product delivery time (25 days) and the maximum consumer's waiting time (7 days).

1. Open the *Main* agent type diagram.

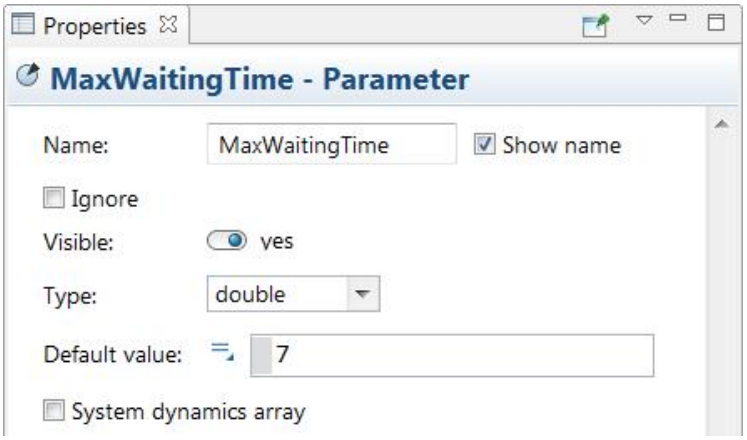
2. Since we don't want the model window to display the model's parameters at runtime, we can place them outside the model window's default display area.

   On *Main*, the model window is depicted with a blue rectangular frame. Elements inside the frame will be visible at the model runtime, but you can hide them by moving the

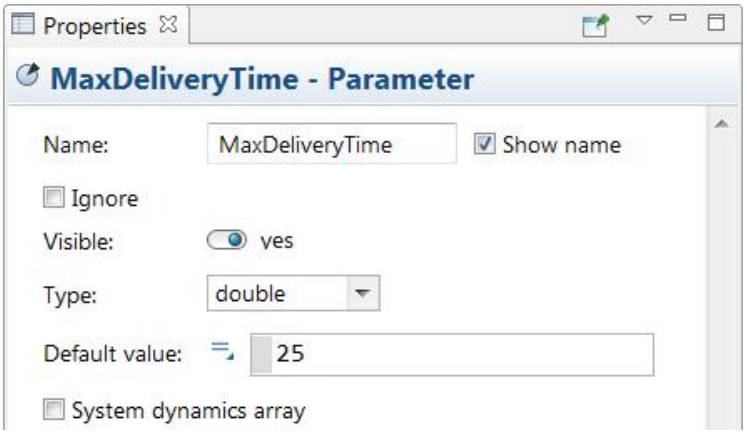
graphical diagram's canvas slightly to the right and placing two parameters as shown in the figure below.



3. Configure the parameters. *MaxWaitingTime* defines the maximum time a consumer will wait for the product (in this case, 7 days).

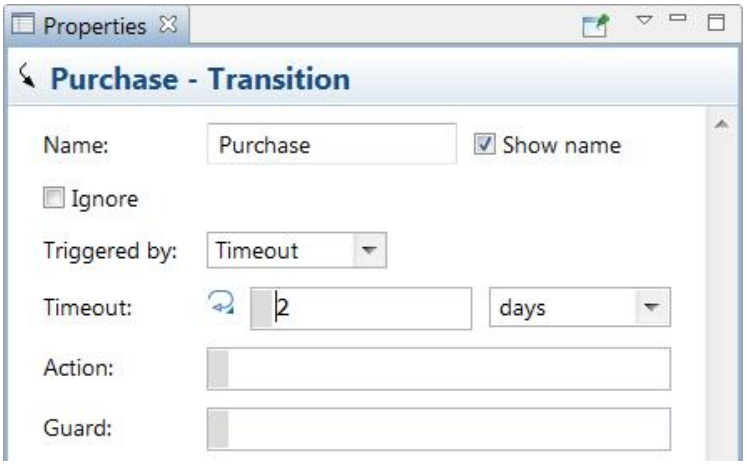


4. Set the other parameter, *MaxDeliveryTime* to 25 days to reflect our assumption it may take up to 25 days to deliver a product.

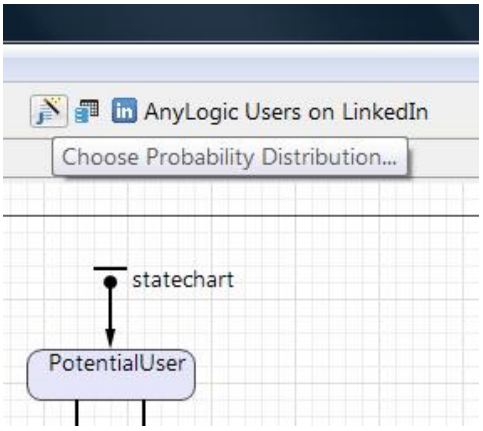


We assume it takes between one and 25 days – with an average of two days – to deliver the product. With that in mind, let's change the delivery time from a fixed two day delivery period to the stochastic expression that describes this pattern.

5. Open the *Consumer* diagram and select the *Purchase* transition. We want to change the transition's timeout expression, and we will do that by using a wizard to choose the distribution function and insert the function's name in the property. To substitute the existing value, use your mouse to select the existing **Timeout** expression.

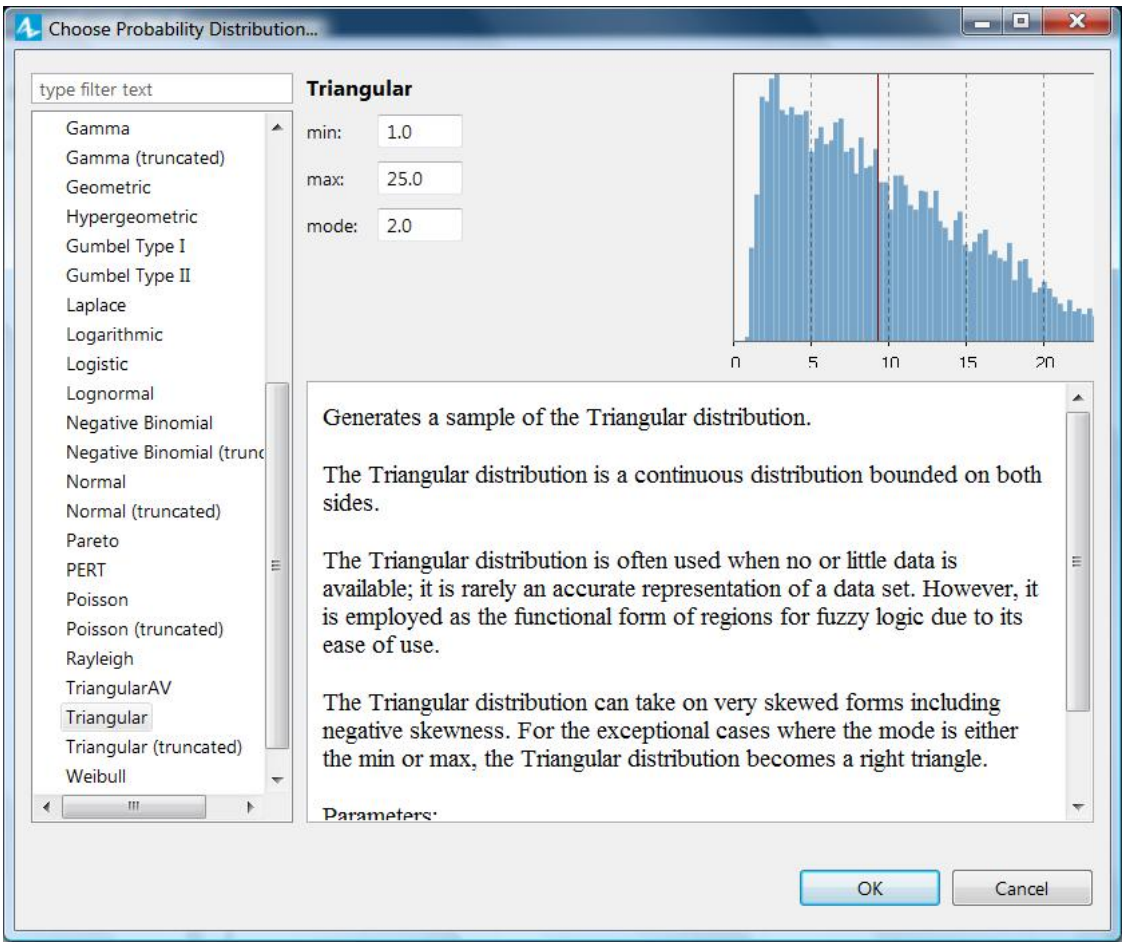


6. Click the **Choose Probability Distribution...** button.



7. You will see the **Choose Probability Distribution...** dialog box.

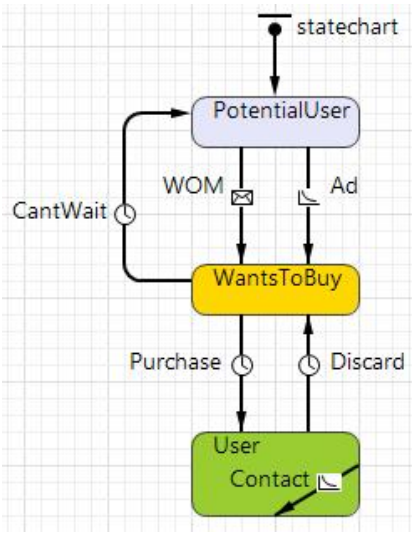
8. The **Choose Probability Description** screen allows you to view the list of supported distributions, and you can click any name in the list to view the distribution's description. Choose *triangular* in the list. Set **min**, **max** and **mode** parameters equal to 1, 25, 2 respectively. In the upper right, you will see PDF instantly built for the distribution with the specified parameters. Click **OK** when finished.



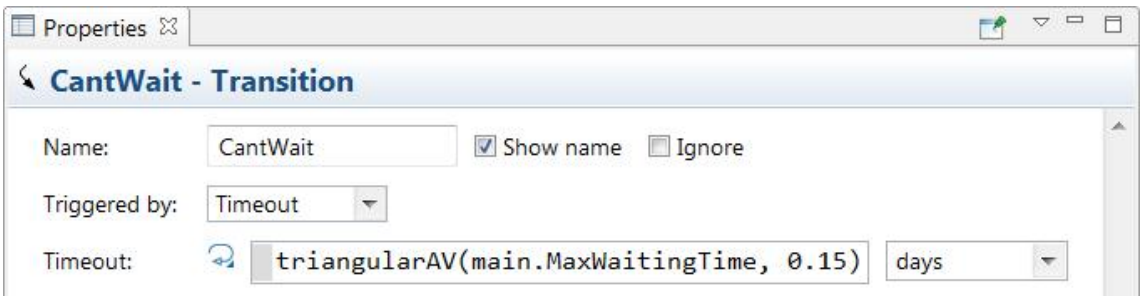
9. You will see the expression *triangular(1, 25, 2)* automatically inserted as the timeout value. Let's modify the line to *triangular(1, main.MaxDeliveryTime, 2).*

   Here *main* is how we access the *Main* agent from the consumer agent.

10. Draw the last transition *CantWait* that goes from *WantsToBuy* to *PotentialUser* state. This transition will model how a consumer's impatience causes them to change their purchase decision, and the *Consumer* diagram will look like this:



11. Modify the transition properties so it is triggered by **Timeout** which equals *triangularAV(main.MaxWaitingTime, 0.15)* **days**

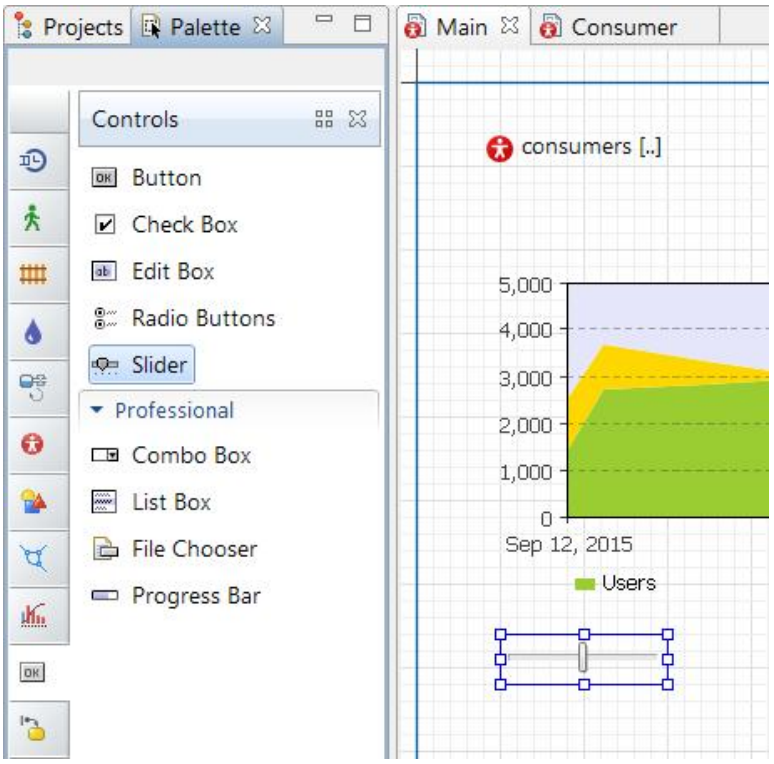


Rather than setting the maximum waiting time equal to constant *MaxWaitingTime*, we assume it follows a triangular distribution with an average of one week and a possible variation to up to 15 percent.

We could easily define maximum waiting time and maximum delivery time as constant parameters, but we want to vary these numbers dynamically and see how these changes affect the system's behavior. One way we can add interactivity to our model is by adding controls and linking them to the model parameters.

We will add a slider control that will let us select a numeric value within a bounded interval. Sliders are often used to modify values of numeric variables and parameters.

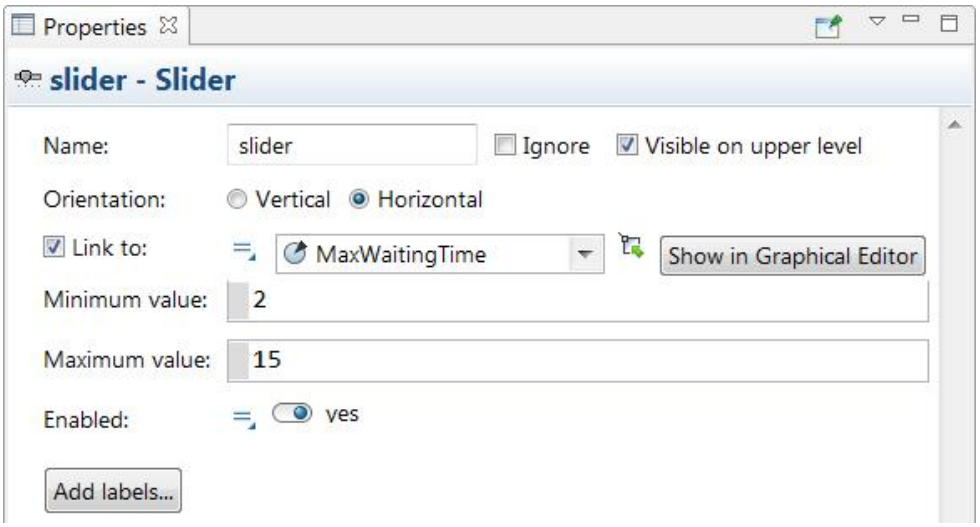
12. Go back to *Main* diagram. Open the **Controls** palette and drag two **Sliders** on to the diagram below the chart. We will eventually link the sliders to our two parameters.



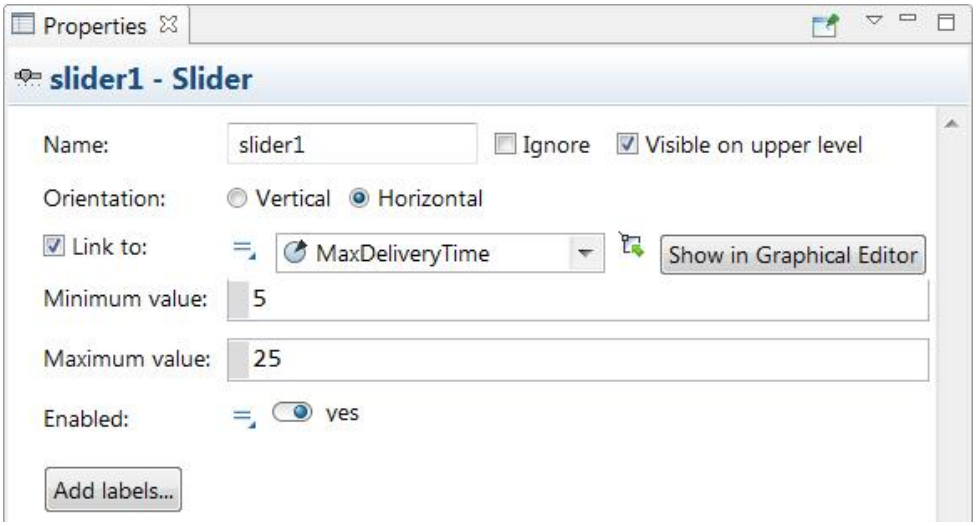
13. Modify the slider's properties:

- Select the checkbox **Link** to and select the parameter *MaxWaitingTime* to the right.
- Set the slider's **Minimum** and **Maximum values**. The parameter value can vary within the range you define here, and we will set 2 as **Minimum** and 15 as **Maximum value**.

- Finally, click the **Add labels...** button to display the slider's minimum, maximum, and current values at runtime (the *min*, *value*, and *max* text shapes will appear beneath the slider).



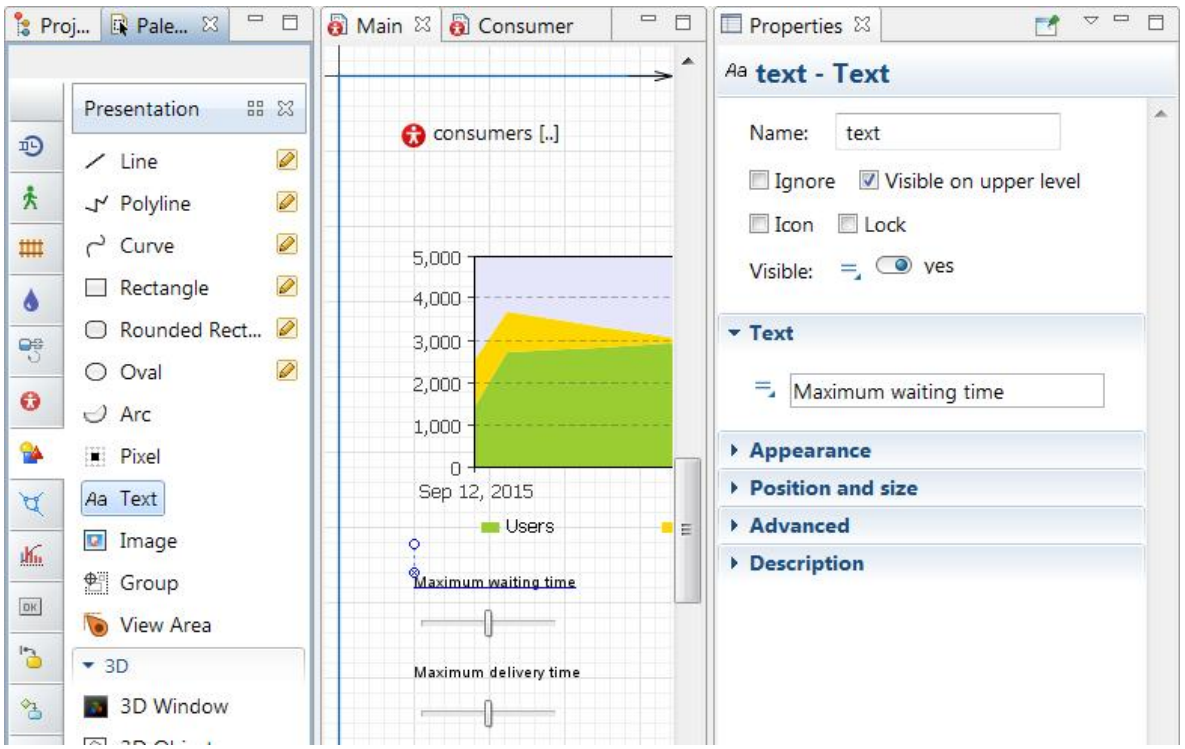
14. Add another slider below and configure it as follows:



Some controls have built-in labels, but you should use the **Text** shape to manually create them for the sliders.

15. Open the **Presentation** palette, drag two **Text** shapes on to the diagram, and place them above the sliders. Let's configure the titles of these controls.

16. In the properties view, in the **Text** section, enter the text that the model will display. Using text shapes, name one slider *Maximum waiting time* and the other slider *Maximum delivery time*.

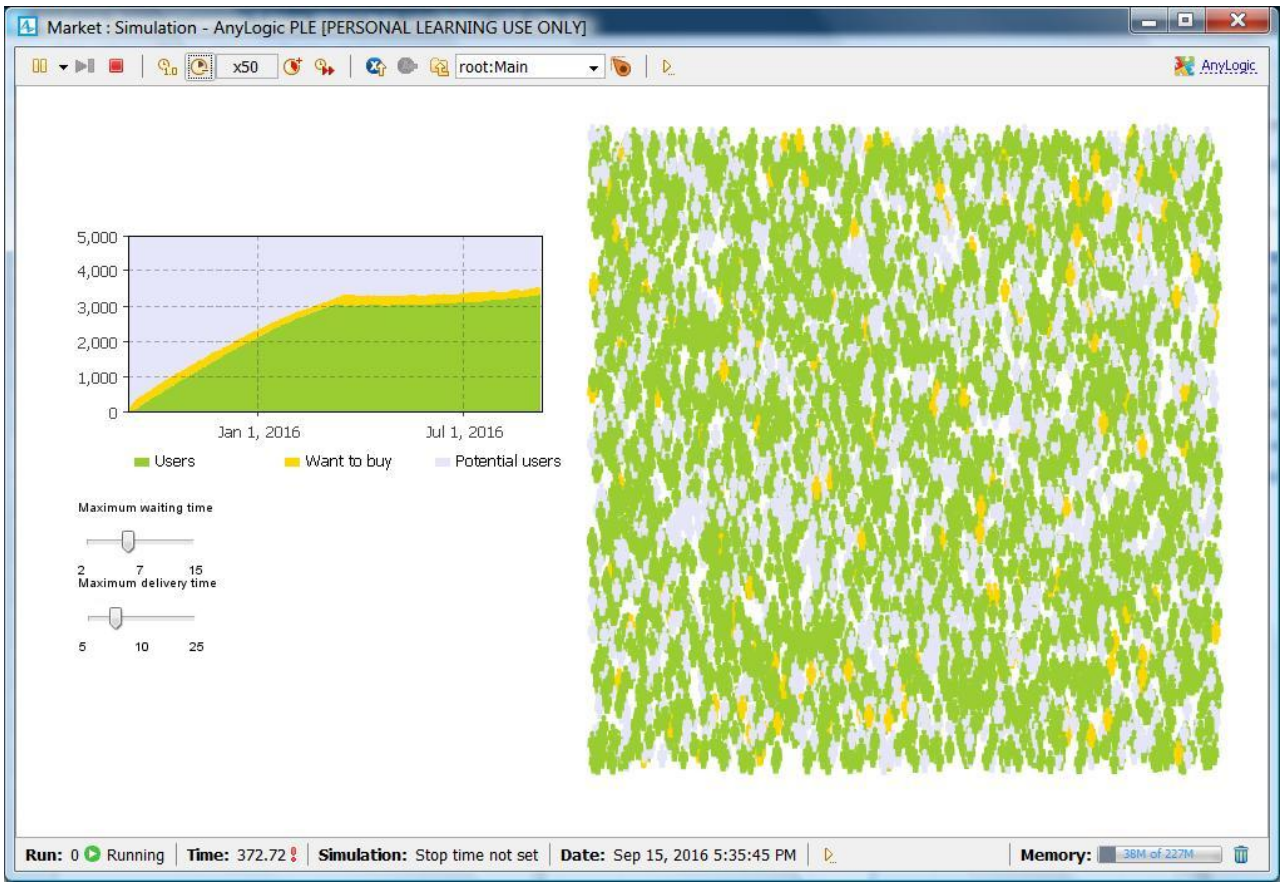


17. In the properties section, under **Appearance**, you can customize the text's color, alignment, font, and point size.

    The labels that display the slider's minimum, current, and maximum values are also Text shapes. Their dynamic properties will display the slider's minimum, current and maximum values at the model's runtime, and you can edit their labels like you would edit any text shape.

    You can also move the *consumers* element to the left beyond the presentation window frame.

18. Run the model and observe the behavior. As you use the sliders to change the maximum waiting time or delivery time, you will see your changes reflected in consumer behaviors and whole adoption dynamics.


Market : Simulation - AnyLogic PLE [PERSONAL LEARNING USE ONLY]
x50
root:Main
AnyLogic
5,000
4,000
3,000
2,000
1,000
0
Jan 1, 2016
Jul 1, 2016
Users
Want to buy
Potential users
Maximum waiting time
2
7
15
Maximum delivery time
5
10
25
Run: 0 Running
Time: 372.72
Simulation: Stop time not set
Date: Sep 15, 2016 5:35:45 PM
Memory:
38M of 227M